Code

Analiza skupa podataka Marketing & Sales

Autori: Danilo Novaković 101/2018, Luka Jevtić 64/2017

1 Uvod

Analiza i predviđanje prihoda od prodaje danas predstavljaju ključni deo uspešnog poslovanja, jer omogućavaju kompanijama da pametnije raspodele marketinške budžete i donose bolje finansijske odluke. Umesto oslanjanja na subjektivne procene i staru praksu, savremeni pristupi koriste podatke i napredne algoritme kako bi se tačno predvideli budući rezultati na osnovu ulaganja, ponašanja kupaca i kanala prodaje.

Za potrebe ovog projekta korišćen je R programski jezik, koji se pokazao kao izvanredan alat za statističku obradu, kreiranje modela mašinskog učenja i vizuelnu analizu podataka, što nam omogućava da na jasan i efikasan način izvedemo ključne poslovne zaključke.

2 Opis podataka

U ovom projektu analizira se skup podataka Marketing & Sales koji obuhvata 60.000 instanci prikupljenih iz poslovanja u MENA regionu (Alexandria,Amman,Cairo,Casablanca,Dubai,Kuwait,Riyadh). Skup sadrži sveobuhvatne informacije o marketinškim budžetima, troškovima online i offline oglašavanja, kanalima prodaje, segmentima kupaca.

Skup podataka preuzet je sa platforme Kaggle:

Izvor: Marketing i Sales Dataset (https://www.kaggle.com/datasets/abdelfattahibrahim/marketing-sales-dataset)

Cilj rada nam je detaljna analiza faktora koji utiču na ostvareni prihod od prodaje (sales_revenue_usd) i razvoj regresionih modela mašinskog učenja za preciznu predikciju prihoda na osnovu marketinških investicija i parametara ponašanja kupaca.

2.1 Učitavanje i dimenzije skupa podataka

Na početku ćemo učitati set podataka

Hide

```
dataset=read.csv("marketing_sales_dataset.csv",header = T)
```

Hide

```
print(paste("Broj kolona je:", ncol(dataset)))
print(paste("Broj redova je:", nrow(dataset)))
```

2.2 Pregled nekoliko redova,naziva feature-a, i njihova struktura

Nakon što smo učitali podatke možemo da proverimo kojih je dimenzija set podataka, vidimo da imamo 60 000 vrsta i 23 kolona.

U nastavku smo pomoću funkcije head koja izlistava prvih nekoliko vrsta našeg set-a podataka a names nazive feature-a.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
head(dataset)
```

Hide

```
names(dataset)
```

Funkcijom str možemo da proverimo kog su tipa naši feature-i i vidimo da ih imamo 23 od čega 7 numeričke (integer - int),10 decimalnih (numeric - num) i 6 karakternih (chr)

Hide

```
str(dataset)
```

Sada nakon što smo videli koji sve feature-i postoje u setu podataka možemo da napišemo kratak opis o njima i da lepo to predstavimo tabelarno,za to nam treba da učitamo paket knitr i iskorisitmo funkciju kable.

2.3 Pregled i opis svih feature-a

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
opis_kolona=data.frame(
  Kolona= c(
    "id", "date", "region", "sales_channel", "product_category",
    "customer_segment", "season", "marketing_budget_usd", "ad_spend_online_usd",
    "ad_spend_offline_usd", "num_promotions", "discount_percentage",
    "num_sales_representatives", "customer_age", "customer_satisfaction_score",
    "competitor_price_index", "website_traffic", "conversion_rate",
    "email_open_rate", "social_media_followers", "days_since_last_purchase",
    "num_previous_purchases", "sales_revenue_usd"
  ),
  Opis = c(
    "Jedinstveni identifikacioni broj transakcije.",
    "Datum izvršene transakcije.",
    "Geografski region u okviru MENA tržišta (Alexandria, Amman, Cairo, Casablanca, Dubai, Kuwait,
Riyadh).",
    "Kanal preko kojeg je obavljena prodaja (npr. Online, Retail Store).",
    "Kategorija proizvoda (Cosmetics, Electronics, Food & Beverage, itd.).",
    "Tip ili segment kupca (Regular, New, Corporate, VIP).",
    "Sezonski period / kvartal u godini (Q1, Q2, Q3, Q4).",
    "Ukupan opredeljeni marketinški budžet u dolarima.",
    "Finansijska sredstva uložena u online oglašavanje.",
    "Finansijska sredstva uložena u offline marketinške kampanje.",
    "Broj aktivnih promotivnih ponuda ili akcija tokom transakcije.",
    "Procenat odobrenog popusta .",
    "Broj angažovanih prodajnih predstavnika.",
    "Starost kupca u godinama.",
    "Ocena zadovoljstva kupca na skali od 1 do 5.",
    "Indeks cene konkurencije u odnosu na naš proizvod.",
    "Saobraćaj / posete veb-sajtu.",
    "Stopa konverzije posetilaca u kupce.",
    "Procenat otvaranja promotivnih mejlova .",
    "Broj pratilaca brenda na društvenim mrežama.",
    "Broj proteklih dana od poslednje kupovine istog kupca.",
    "Ukupan broj prethodnih kupovina koje je kupac ostvario.",
    "Ostvareni prihod od prodaje u dolarima (ciljna promenljiva - target)."
  )
)

knitr::kable(opis_kolona, col.names = c("Naziv kolone", "Opis promenljive"))
```

2.4 Deskriptivna statistika i analiza ciljne promenljive

Nakon što smo definisali šta nam predstavlja svaki feature u skupu podataka, možemo da koristimo funkciju summary() kako bismo dobili osnovne statističke pokazatelje za sve naše promenljive,ali posebnu pažnju moramo da posvetimo našoj ciljnoj promenljivoj (sales_revenue_usd) koja predstavlja ostvareni prihod od prodaje u američkim dolarima.Njena distribucija ključna za kasnije modelovanje

Hide

```
summary(dataset)
```

Što se tiče naše ciljne promenljive (sales_revenue_usd), iz priloženih statistika se lepo vidi da je prosečna vrednost od oko 5.911 dolara viša od medijane koja iznosi 4.340 dolara. To nam odmah govori da imamo nekoliko transakcija sa prilično velikim prihodima koje vuku prosek nagore. Većina prodaja se vrti oko nižih i srednjih iznosa (pa nam se tako 75% podataka nalazi ispod 7.091 dolar), dok samo manji broj transakcija ide u neke ekstremnije visine, sve do maksimalnih 190.377 dolara.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

2.5 Provera nedostajućih vrednosti

Hoćemo da identifikujemo nedostajuće vrednosti možemo da koristimo funkciju colSums i vidimo da email_open_rate ima 1790,discount_percentage ima 1808 ,customer_satisfaction_score ima 1844 i days_since_last_purchase ima 1836 nedostajućih vrednosti.

Hide

```
colSums(is.na(dataset))
```

2.6 Provera duplikata i unikatnih vrednosti

U ovom koraku ćemo proveriti da li u našem setu podataka postoje duplikati, odnosno da li se neki redovi u potpunosti ponavljaju.I vidimo na osnovu rezultata da ih nema i da je svaka od 60.000 instanci jedinstvena.

Hide

```
total_duplicate_rows = sum(duplicated(dataset))
print(paste("Ukupan broj dupliranih redova u dataset-u:", total_duplicate_rows))
```

Sada ćemo proveriti koje sve unikatne vrednosti imamo u našim kategorijskim kolonama, da vidimo da li tu ima nekih čudnih unosa ili slovnih grešaka. Za to možemo da pregledamo unikatne vrednosti za ključne tekstualne promenljive:

Hide

```
sapply(dataset[sapply(dataset, is.character)], function(x) length(unique(x)))
```

Vidimo da date ima 1461,region ima 7 ,sales_channel ima 5,product_category ima 5,customer_segment ima 4 i season ima 4 jedinstvene vrednosti

U ovom koraku ćemo proveriti kako su podaci raspoređeni po kategorijama, odnosno kolika je frekvencija opservacija u ključnim kategorijskim kolonama:

Hide

```
table(dataset$region)
table(dataset$sales_channel)
table(dataset$product_category)
table(dataset$customer_segment)
table(dataset$season)
```

Kao što vidimo iz tabela frekvencija, opservacije su prilično ravnomerno raspoređene po različitim regionima, kanalima prodaje, kategorijama proizvoda i sezonama, što nam govori da nemamo ekstremne disbalanse u uzorku.

2.7 Provera validnosti opsega i logičkih pravila

U ovom koraku ćemo proveriti opsege minimalnih i maksimalnih vrednosti za sve numeričke promenljive, kako bismo utvrdili osnovne granice u kojima se podaci kreću:

Hide

```
sapply(dataset[sapply(dataset, is.numeric)], function(x) c(Min = min(x, na.rm = TRUE), Max = max
(x, na.rm = TRUE)))
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Nakon provere opsega, analizirane su minimalne i maksimalne vrednosti da se utvrdi da li ima nekih nelogičnosti. Uočeno je da se svi podaci kreću u normalnim granicama: godine kupaca su od 18 do 74, ocene zadovoljstva su između 2 i 5, a maksimalni popust je 40%. Budžeti i troškovi jesu visoki, ali su u skladu sa realnim poslovanjem i nema negativnih cena niti nelogičnih unosa. Samim tim, numerički opsezi su potpuno ispravni.

U ovom koraku ćemo proveriti da li u setu podataka postoje logičke nepravilnosti:

Provera da li popust izlazi van očekivanog opsega [0%, 100%]

Hide

```
sum(dataset$discount_percentage < 0 | dataset$discount_percentage > 100, na.rm = TRUE)
```

Provera da li ocene zadovoljstva izlaze van opsega [1, 5]

Hide

```
sum(dataset$customer_satisfaction_score < 1 | dataset$customer_satisfaction_score > 5, na.rm = TRU
E)
```

Provera da li broj dana od poslednje kupovine prelazi 365

Hide

```
sum(dataset$days_since_last_purchase < 0 | dataset$days_since_last_purchase > 365, na.rm = TRUE)
```

2.8 Analiza outlier-a

U ovom koraku ćemo identifikovati potencijalne outlier-e za ključne numeričke promenljive u setu podataka (prihode, marketing budžet, web saobraćaj i društvene mreže) pomoću importovanja tidiverse paketa i njegovog korišćenja za iscrtavnja grafika:

Hide

```
ggplot(dataset, aes(x = "", y = sales_revenue_usd)) +
  geom_boxplot(fill = "#89CFF0", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5,
outlier.shape = 16) +
  theme_minimal() +
  labs(title = "Boxplot - Sales Revenue", x = "Sales Revenue", y = "USD") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```

Kod prihoda od prodaje (sales_revenue_usd), Boxplot dijagram prikazuje znatnu koncentraciju tačaka iznad gornje granice dijagrama. Ove vrednosti predstavljaju legitimne visoke transakcije ostvarene na uspešnijim tržištima i ne predstavljaju greške u unosu, te se zadržavaju u analizi.

Hide

```
ggplot(dataset, aes(x = "", y = marketing_budget_usd)) +
  geom_boxplot(fill = "#77DD77", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5,
outlier.shape = 16) +
  theme_minimal() +
  labs(title = "Boxplot - Marketing Budget", x = "Marketing Budget", y = "USD") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```

Slična situacija je i kod marketing budžeta (marketing_budget_usd), gde se uočava gust skup vrednosti iznad gornjeg kvartila. Reč je o realnim poslovnim odlukama za obimnije reklamne kampanje, pa se ove opservacije takođe zadržavaju u skupu.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Hide

```
ggplot(dataset, aes(x = "", y = website_traffic)) +
  geom_boxplot(fill = "#FFB347", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5,
outlier.shape = 16) +
  theme_minimal() +
  labs(title = "Boxplot - Website Traffic", x = "Website Traffic", y = "Visits") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```

Grafik web saobraćaja (website_traffic) prikazuje značajan broj ekstremnijih vrednosti usmerenih ka gornjoj granici. One odražavaju periode izuzetno visoke posete sajtu tokom uspešnih kampanja i predstavljaju stvarne podatke bez potrebe za uklanjanjem.

Hide

```
ggplot(dataset, aes(x = "", y = social_media_followers)) +
  geom_boxplot(fill = "#CCCCFF", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5,
outlier.shape = 16) +
  theme_minimal() +
  labs(title = "Boxplot - Social Media Followers", x = "Social Media Followers", y = "Count") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```

Kod broja pratilaca na društvenim mrežama (social_media_followers) prisutne su vertikalno raspršene tačke ka samom vrhu skale, što je sasvim očekivano u praksi za brendove koji ostvaruju veći organski domet ili viralne objave.

Zaključak: Za sve posmatrane promenljive jeste da prisutni outlieri nemaju karakter grešaka u merenju, već su prirodan odraz varijacija u realnom poslovnom okruženju, te se zadržavaju za dalju obradu.

2.9 Analiza gustine distribucije

U ovom koraku ćemo proveriti distribuciju ključnih numeričkih promenljivih pomoću grafika gustine (density plot) kako bismo utvrdili da li podaci prate prirodne obrasce ili pokazuju znakove veštačkog generisanja:

Hide

```
ggplot(dataset, aes(x = sales_revenue_usd)) +
  geom_density(fill = "#3498DB", color = "#2C3E50", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Density Plot: Sales Revenue", x = "Sales Revenue (USD)", y = "Density") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Distribucija prihoda od prodaje jasno pokazuje izraženu desnu asimetriju sa dugim desnim repom. Ogromna većina transakcija donosi niže i umerene iznose prihoda, dok se tek manji broj velikih prodaja pruža ka desnom kraju skale. Kriva je glatka i kontinuirana, što je sasvim očekivano za realne finansijske podatke i jasno nam ukazuje na potrebu za logaritamskom transformacijom u nastavku rada.

Hide

```
ggplot(dataset, aes(x = marketing_budget_usd)) +
  geom_density(fill = "#27AE60", color = "#2C3E50", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Density Plot: Marketing Budget", x = "Marketing Budget (USD)", y = "Density") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Kod marketing budžeta vidimo veoma sličan šablon — kriva ima jedan dominantan pik na manjim iznosima i postepeno opada kako budžeti rastu. Ovakav raspored odslikava uobičajenu praksu gde se većina kampanja finansira umerenim sredstvima, dok se znatno ređe odobravaju izuzetno visoki budžeti.

Hide

```
ggplot(dataset, aes(x = website_traffic)) +
  geom_density(fill = "#F39C12", color = "#2C3E50", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Density Plot: Website Traffic", x = "Website Traffic (Visits)", y = "Density") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Kriva saobraćaja na sajtu je takođe pozitivno asimetrična, ali potpuno glatka. Većina dana beleži standardnu posećenost, dok manji broj uspešnih promocija ili kampanja generiše veliku posetu. Na grafiku nema neprirodnih skokova ili rupa, što potvrđuje stabilnost podataka.

Hide

```
ggplot(dataset, aes(x = social_media_followers)) +
  geom_density(fill = "#9B59B6", color = "#2C3E50", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Density Plot: Social Media Followers", x = "Social Media Followers (Count)", y =
"Density") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Broj pratilaca na društvenim mrežama očekivano pokazuje najveću koncentraciju pri dnu skale. Većina brendova ili naloga ima umerenu publiku, dok manji broj uspeva da privuče stotine hiljada ili milione pratilaca.

Sve posmatrane numeričke promenljive imaju prirodne, glatke krive sa izraženom desnom asimetrijom. Podaci deluju konzistentno i spremni su za naredne korake preprocesiranja i transformacije.

3 Analiza nedostajućih vrednosti

3.1 Uvod u analizu nedostajućih vrednosti

U ovoj fazi analiziraju se nedostajuće vrednosti u skupu podataka. Cilj je utvrditi u kojim kolonama postoje nedostajuće vrednosti, koliki je njihov procenat, da li se nedostajuće vrednosti pojavljuju u više kolona istovremeno i da li postoji obrazac njihovog pojavljivanja. Takođe se ispituje povezanost nedostajućih vrednosti sa drugim posmatranim promenljivama i ciljnom promenljivom `sales_revenue_usd` .

Na osnovu sprovedene analize proceniće se da li je pretpostavka MCAR (*Missing Completely At Random*) prihvatljiva. Nakon toga biće razmotreno nekoliko strategija za rešavanje nedostajućih vrednosti: imputacija medianom i modom, višestruka imputacija pomoću MICE algoritma sa *Predictive Mean Matching* (PMM) metodom.

3.2 Broj nedostajućih vrednosti po kolonama

Prvi korak predstavlja određivanje broja nedostajućih vrednosti u svakoj koloni. Na ovaj način možemo identifikovati promenljive koje zahtevaju dodatnu obradu i utvrditi obim problema sa nedostajućim podacima.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
missing_counts = colSums(is.na(dataset))
missing_summary = data.frame(
  Kolona = names(missing_counts),
  Broj_missing = as.numeric(missing_counts)
)
missing_summary = missing_summary[missing_summary$Broj_missing > 0, ]
missing_summary
```

3.3 Procenat nedostajućih vrednosti

Pored apsolutnog broja nedostajućih vrednosti, izračunat je i njihov procenat u odnosu na ukupan broj redova.

Hide

```
missing_summary$Procenat_missing = round(
  (missing_summary$Broj_missing / nrow(dataset)) * 100,
  2
)
missing_summary
```

U posmatranom skupu podataka nedostajuće vrednosti se pojavljuju u nekoliko numeričkih promenljivih, sa udelom od približno 3% po promenljivoj.

3.4 Vizuelizacija nedostajućih vrednosti

Radi lakšeg uočavanja razlika između promenljivih, procenat nedostajućih vrednosti prikazan je grafički.

Hide

```
ggplot(missing_summary, aes(x = reorder(Kolona, Procenat_missing), y = Procenat_missing)) +
  geom_col(fill = "#3498db", color = "#2980b9") +
  coord_flip() +
  labs(
    title = "Procenat nedostajućih vrednosti po kolonama",
    x = "Kolona",
    y = "Nedostajuće vrednosti (%)"
  ) +
  theme_minimal()
```

3.5 Identifikovanje redova sa više nedostajućih vrednosti

Sledeći korak je analiza raspodele nedostajućih vrednosti po redovima. Posebno je važno utvrditi da li se nedostajuće vrednosti uglavnom pojavljuju pojedinačno ili postoji veći broj redova u kojima nedostaje više atributa istovremeno.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
missing_per_row = rowSums(is.na(dataset))
table(missing_per_row)

rows_multiple_missing = dataset[missing_per_row >= 2, ]
num_multiple = nrow(rows_multiple_missing)
pct_multiple = round(mean(missing_per_row >= 2) * 100, 2)

print(paste("Broj redova sa najmanje dve missing vrednosti:", num_multiple))
print(paste("Procenat redova sa najmanje dve missing vrednosti:", pct_multiple, "%"))
```

Dobijeni rezultati pokazuju koliko se često više nedostajućih vrednosti pojavljuje u okviru istog reda. Ova analiza je značajna prilikom izbora strategije imputacije, jer veći broj istovremeno nedostajućih vrednosti može smanjiti količinu informacija dostupnih za pouzdanu imputaciju.

3.6 Analiza obrazaca nedostajućih vrednosti

Pored broja nedostajućih vrednosti, analiziran je i njihov obrazac pojavljivanja. Cilj je utvrditi koje promenljive najčešće imaju nedostajuće vrednosti zajedno.

Hide

```
missing_variables = c(
  "discount_percentage",
  "customer_satisfaction_score",
  "email_open_rate",
  "days_since_last_purchase"
)

op = par(mar = c(6, 5, 10, 4), cex = 0.45)

mice::md.pattern(
  dataset[, missing_variables],
  plot = TRUE,
  rotate.names = TRUE
)

par(op)
```

Na osnovu prikazanog obrasca može se uočiti da se nedostajuće vrednosti u analiziranim promenljivama pojavljuju uglavnom pojedinačno. Ovaj rezultat će biti uzet u obzir prilikom izbora metode imputacije.

3.7 Indikatori nedostajućih vrednosti

Kako bi originalni skup podataka ostao nepromenjen, za dalju analizu nedostajućih vrednosti kreirana je posebna kopija skupa podataka pod nazivom missingness_data. U ovoj kopiji kreiraju se pomoćne binarne promenljive koje označavaju da li određena vrednost nedostaje. Vrednost 1 označava da podatak nedostaje, dok vrednost 0 označava da je podatak dostupan. Originalni dataset se ne menja, čime se omogućava njegovo kasnije korišćenje za različite strategije obrade podataka.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
missingness_data = dataset %>%
  mutate(
    missing_discount = as.integer(
      is.na(discount_percentage)
    ),

    missing_satisfaction = as.integer(
      is.na(customer_satisfaction_score)
    ),

    missing_email = as.integer(
      is.na(email_open_rate)
    ),

    missing_days = as.integer(
      is.na(days_since_last_purchase)
    )
  )
```

3.8 Test nezavisnosti missingness-a

U nastavku se ispituje da li je pojavljivanje nedostajućih vrednosti povezano sa kategorijskim promenljivama. Analizirane su promenljive region, sales_channel, product_category, customer_segment i season. Za ispitivanje povezanosti koristi se χ^2 test nezavisnosti.

Nulta hipoteza testa je da su pojavljivanje nedostajuće vrednosti i posmatrana kategorijska promenljiva nezavisni. Kao nivo statističke značajnosti koristi se $\alpha = 0,05$.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
categorical_variables = c(
  "region",
  "sales_channel",
  "product_category",
  "customer_segment",
  "season"
)

missing_indicators = c(
  "missing_discount",
  "missing_satisfaction",
  "missing_email",
  "missing_days"
)

missing_names = c(
  "discount_percentage",
  "customer_satisfaction_score",
  "email_open_rate",
  "days_since_last_purchase"
)

chi_results = data.frame(
  Missing_variable = character(),
  Categorical_variable = character(),
  Chi_squared = numeric(),
  p_value = numeric(),
  Zakljucak = character()
)

for (i in seq_along(missing_indicators)) {

  for (categorical_variable in categorical_variables) {

    tabela = table(
      missingness_data[[missing_indicators[i]]],
      missingness_data[[categorical_variable]]
    )

    test = chisq.test(tabela)

    zakljucak = ifelse(
      test$p.value < 0.05,
      "Statistički značajna povezanost",
      "Nema statistički značajne povezanosti"
    )

    chi_results = rbind(
      chi_results,
      data.frame(
        Missing_variable = missing_names[i],
        Categorical_variable = categorical_variable,
        Chi_squared = as.numeric(test$statistic),
        p_value = test$p.value
      )
    )
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
  }
}

chi_results
```

Pošto je p-vrednost veća ili jednaka nivou značajnosti α = 0,05, nema dovoljno statističkih dokaza za odbacivanje nulte hipoteze. Nije utvrđena statistički značajna povezanost između pojavljivanja nedostajućih vrednosti i posmatrane kategorijske promenljive.

3.9 Poređenje ciljne promenljive

Kako bi se proverilo da li se prihod od prodaje razlikuje između redova sa i bez nedostajućih vrednosti, analizirana je ciljna promenljiva sales_revenue_usd. Za svaku promenljivu izračunate su srednja i medijalna vrednost prihoda za slučajeve kod kojih podatak nedostaje i slučajeve kod kojih je dostupan.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
target_comparison = data.frame(
  Missing_variable = character(),
  Status = character(),
  Number_of_rows = integer(),
  Mean_revenue = numeric(),
  Median_revenue = numeric()
)

for (variable in missing_variables) {

  indicator_name = switch(
    variable,
    discount_percentage = "missing_discount",
    customer_satisfaction_score = "missing_satisfaction",
    email_open_rate = "missing_email",
    days_since_last_purchase = "missing_days"
  )

  status =ifelse(
    missingness_data[[indicator_name]] == 1,
    "Missing",
    "Available"
  )

  temp = data.frame(
    Status = status,
    Revenue = missingness_data$sales_revenue_usd
  )

  summary_temp = temp %>%
    group_by(Status) %>%
    summarise(
      Number_of_rows = n(),
      Mean_revenue = mean(Revenue),
      Median_revenue = median(Revenue)
    )

  summary_temp$Missing_variable = variable

  target_comparison = rbind(
    target_comparison,
    summary_temp[, c(
      "Missing_variable",
      "Status",
      "Number_of_rows",
      "Mean_revenue",
      "Median_revenue"
    )]
  )
}

target_comparison
```

3.10 Boxplot poređenje targeta

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Pored numeričkog poređenja, distribucije prihoda za redove sa i bez nedostajućih vrednosti prikazane su pomoću boxplot grafika. Pošto ciljna promenljiva sales_revenue_usd ima izraženu desnu asimetriju i ekstremne vrednosti, koristi se logaritamska skala radi preglednijeg prikaza.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
target_plot_data = data.frame(
  sales_revenue_usd = missingness_data$sales_revenue_usd
)

for (variable in missing_variables) {

  indicator_name = switch(
    variable,
    discount_percentage = "missing_discount",
    customer_satisfaction_score = "missing_satisfaction",
    email_open_rate = "missing_email",
    days_since_last_purchase = "missing_days"
  )

  target_plot_data[[variable]] = ifelse(
    missingness_data[[indicator_name]] == 1,
    "Missing",
    "Available"
  )
}

target_plot_data_long = target_plot_data %>%
  pivot_longer(
    cols = all_of(missing_variables),
    names_to = "Missing_variable",
    values_to = "Status"
  )

ggplot(
  target_plot_data_long,
  aes(
    x = Status,
    y = sales_revenue_usd,
    fill = Status
  )
) +
  geom_boxplot(
    color = "#2C3E50",
    outlier.color = "#E74C3C",
    outlier.size = 1.5,
    outlier.shape = 16,
    alpha = 0.8
  ) +
  scale_y_log10() +
  facet_wrap(~ Missing_variable) +
  labs(
    title = "Prihod u odnosu na prisustvo nedostajućih vrednosti",
    x = "Status vrednosti",
    y = "Sales revenue (log skala)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    legend.position = "none" # Uklanja legendu ako koristiš fill = Status, pošto su labele već na
x-osi
  )
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Analizom boxplot dijagrama za sve četiri posmatrane varijable (customer_satisfaction_score, days_since_last_purchase, discount_percentage i email_open_rate) uočava se da ne postoji značajna razlika u distribuciji prihoda (Sales revenue) između grupa sa prisutnim (Available) i nedostajućim (Missing) vrednostima. Medijane i interkvartilni rasponi prihoda su gotovo identični u obe grupe za svaku varijablu. Veći broj ekstremnih vrednosti u kategoriji Available posledica je znatno većeg uzorka. Ovi rezultati ukazuju na to da prisustvo nedostajućih vrednosti nije uslovljeno visinom ostvarenog prihoda.

3.11 MCAR analiza

Na osnovu prethodno sprovedenih analiza ispituje se da li je razumno pretpostaviti da nedostajuće vrednosti nastaju potpuno nasumično, odnosno da li je MCAR (Missing Completely At Random) pretpostavka prihvatljiva. Pri tome se ne oslanjamo samo na jedan rezultat, već kombinujemo analizu obrazaca nedostajućih vrednosti, testove nezavisnosti i poređenje ciljne promenljive.Za dodatnu formalnu proveru koristi se MCAR test iz paketa MICE.

Hide

```
# 1. Filtriramo samo numeričke kolone i uklanjamo 'id' ako je ostao
mcar_data = dataset %>%
  select(where(is.numeric)) %>%
  select(-any_of("id"))

# 2. Uzimamo reprezentativni uzorak od 5000 redova radi uštede memorije
set.seed(123)
mcar_data_sample = mcar_data %>%
  slice_sample(n = 5000)

# 3. Pokretanje brze MICE imputacije za test
mcar_imp = mice(
  mcar_data_sample,
  m = 3,
  method = "pmm",
  maxit = 3,
  printFlag = FALSE,
  seed = 123
)

# 4. Pokretanje MCAR testa
mcar_result = mice::mcar(
  mcar_data_sample,
  imputed = mcar_imp,
  replications = 200,
  alpha = 0.05
)

# Prikaz rezultata
mcar_result
```

Formalni Hawkins test je pokazao statističku značajnost usled narušene višedimenzionalne normalnosti podataka. Međutim, Anderson-Darling rank test, koji ne zavisi od pretpostavke normalnosti, nije pokazao statističku značajnost ($p = 0.495$). Kako prethodni χ^2 testovi i poređenja distribucije ciljne promenljive takođe ne ukazuju na povezanost nedostajanja sa posmatranim varijablama, MCAR (Missing Completely At Random) se prihvata kao razumna radna pretpostavka. Pored toga, primenjena MICE/PMM metoda ostaje u potpunosti metodološki opravdana i u slučaju šireg MAR (Missing At Random) mehanizma.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

3.12 Strategija imputacije nedostajućih vrednosti (MICE / PMM)

Nakon analize nedostajućih vrednosti, kao jedinstvena strategija za obradu izabrana je napredna metoda MICE sa PMM (Predictive Mean Matching) algoritmom.Jednostavnije strategije unapred su odbačene iz metodoloških razloga:

Complete-case analiza (uklanjanje redova): dovela bi do nepotrebnog gubitka značajnog dela skupa podataka.

Imputacija medijanom ili srednjom vrednošću: veštački bi smanjila varijansu i narušila prirodne korelacije između atributa.

Hide

```
# Uklanjamo identifikatore i ne-numeričke atribute koji nisu pogodni za PMM
mice_data = dataset %>%
  select(
    -id,
    -date
  )
set.seed(123)
mice_model = mice(
  mice_data,
  m = 5,
  method = "pmm",
  maxit = 5,
  printFlag = FALSE,
  seed = 123
)
#data set bez missing values
dataset_clean=complete(mice_model, 1)

#vracanje id i date
dataset_clean$id = dataset$id
dataset_clean$date = dataset$date

#Reorganizacija kolona da budu u originalnom redosledu
dataset_clean = dataset_clean %>%
  select(colnames(dataset))

#Provera: Potvrda da nema preostalih NA vrednosti
colSums(is.na(dataset_clean))
```

4 Feature engineering

U ovom poglavlju kreiramo nove feature na osnovu postojećih podataka kako bismo modelima mašinskog učenja pružili dodatne ulazne informacije.

Kreirani su sledeći feature-i:

4.1 Kreiranje finansijskih i marketinških pokazatelja

total_ad_spend: Predstavlja koliko je novca ukupno otišlo na reklame, kada saberemo online i offline troškove.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
dataset_clean$total_ad_spend = dataset_clean$ad_spend_online_usd + dataset_clean$ad_spend_offline_usd
```

online_share: Pokazuje koliki udeo u ukupnom budžetu za reklame zauzima baš online oglašavanje, što nam otkriva koliko je neka kampanja digitalno orijentisana.

Hide

```
dataset_clean$online_share = ifelse(dataset_clean$total_ad_spend > 0, dataset_clean$ad_spend_online_usd / dataset_clean$total_ad_spend, 0)
```

budget_utilization: Pokazuje nam koliki deo ukupno namenjenog budžeta je stvarno potrošen na reklame.

Hide

```
dataset_clean$budget_utilization = ifelse(dataset_clean$marketing_budget_usd > 0, dataset_clean$total_ad_spend / dataset_clean$marketing_budget_usd, 0)
```

unspent_budget: Predstavlja ostatak novca koji je bio planiran u budžetu, ali na kraju nije potrošen na oglašavanje.

Hide

```
dataset_clean$unspent_budget = dataset_clean$marketing_budget_usd - dataset_clean$total_ad_spend
```

spend_per_rep: Predstavlja koliko je novca za reklame potrošeno po jednom prodajnom predstavniku, što nam pomaže da vidimo opterećenje i efikasnost tima terenu.

Hide

```
dataset_clean$spend_per_rep = dataset_clean$total_ad_spend / dataset_clean$num_sales_representatives
```

spend_per_promotion: Predstavlja koliko je novca uloženo po jednoj aktivnoj akciji ili popustu.

Hide

```
dataset_clean$spend_per_promotion =ifelse(dataset_clean$num_promotions > 0, dataset_clean$total_ad_spend / dataset_clean$num_promotions, dataset_clean$total_ad_spend)
```

budget_per_traffic: Predstavlja odnos između ukupnog budžeta i broja ljudi koji su posetili sajt.

Hide

```
dataset_clean$budget_per_traffic = dataset_clean$marketing_budget_usd / dataset_clean$website_traffic
```

4.2 Logaritamske transformacije i analiza distribucija

log_budget, log_traffic, log_followers logaritamski transformisane vrednosti budžeta, poseta sajtu i broja pratilaca,čijom se primenom smanjuje visoka asimetrija podataka i ublažava uticaj ekstremnih vrednosti na modele.

Hide

```
dataset_clean$log_budget = log(dataset_clean$marketing_budget_usd)
dataset_clean$log_traffic = log(dataset_clean$website_traffic)
dataset_clean$log_followers = log(dataset_clean$social_media_followers + 1)
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

4.2.1 Analiza distribucija pre i posle transformacije

Hide

```
summary(dataset_clean[, c("marketing_budget_usd", "log_budget",
                    "website_traffic", "log_traffic",
                    "social_media_followers", "log_followers")])
```

Pogled na statistiku pre i posle transformacije odlično ilustruje problem koji smo imali sa originalnim podacima. Kod početnih vrednosti za budžet, posete i pratioce vidimo ogroman raskorak između medijane i proseka (recimo, kod saobraćaja je medijana oko 8.000, a prosek skače na preko 24.000 zbog nekoliko drastično velikih vrednosti na desnom kraju). To znači da su podaci bili prilično razvučeni i asimetrični. Kada smo primenili logaritam prosek i medijana su se praktično poklopili, a ekstremni rasponi su svedeni na normalnu meru. Time smo eliminisali raspon velikih brojeva, pa će modeli moći znatno stabilnije i preciznije da uče na ovim podacima.

4.3 Vremenske karakteristike (Time Features)

month i quarter oznake meseca i kvartala u godini, korisne za praćenje periodičnih skokova u prodaji (poput prazničnih kupovina).

Hide

```
dataset_clean$date = as.Date(dataset_clean$date)
dataset_clean$month = format(dataset_clean$date, "%m")
dataset_clean$quarter = quarters(dataset_clean$date)
```

day_of_week: Dan u nedelji kada je transakcija zabeležena.

Hide

```
dataset_clean$day_of_week = weekdays(dataset_clean$date)
```

is_weekend: Indikator (0 ili 1) koji govori da li je kupovina obavljena tokom vikenda.

Hide

```
dataset_clean$is_weekend =ifelse(dataset_clean$day_of_week %in% c("Saturday", "Sunday"), 1, 0)
```

is_year_end: Pokazuje da li transakcija pada u decembar, period pojačanih prazničnih aktivnosti.

Hide

```
dataset_clean$is_year_end=ifelse(format(dataset_clean$date, "%m") == "12", 1, 0)
```

days_since_start: Linearni trend koji meri koliko je dana prošlo od prvog zabeleženog datuma u setu, hvatajući dugoročni rast ili pad.

Hide

```
dataset_clean$days_since_start = as.numeric(dataset_clean$date - min(dataset_clean$date, na.rm = T
RUE))
```

season: Definiše meteorološku sezonu u kojoj je ostvarena prodaja.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
dataset_clean$season = case_when(
  dataset_clean$month %in% c("12", "01", "02") ~ "Zima",
  dataset_clean$month %in% c("03", "04", "05") ~ "Proleće",
  dataset_clean$month %in% c("06", "07", "08") ~ "Leto",
  dataset_clean$month %in% c("09", "10", "11") ~ "Jesen",
  TRUE ~ "Nepoznato"
)
```

4.4 Karakteristike kupaca (Customer Features)

purchase_frequency: Predstavlja intenzitet kupovine, odnosno prosečnu učestalost kojom kupac obavlja transakcije tokom svog aktivnog staža. Računa se stavljanjem u odnos ukupnog broja prethodnih kupovina i efektivnog perioda aktivnosti, čime dobijamo realniju sliku o vernosti kupca nego da posmatramo samo grubu sumu.

Hide

```
dataset_clean$purchase_frequency = ifelse(
  !is.na(dataset_clean$days_since_last_purchase) & dataset_clean$days_since_last_purchase > 0,
  dataset_clean$num_previous_purchases / dataset_clean$days_since_last_purchase,
  dataset_clean$num_previous_purchases
)
```

is_returning: Logički indikator (0 ili 1) koji jasno razdvaja nove kupce od onih stalnih koji se redovno vraćaju.

Hide

```
dataset_clean$is_returning = ifelse(dataset_clean$customer_segment == "Regular" | dataset_clean$nu
m_previous_purchases > 1, 1, 0)
```

customer_age_group: Pretvara numeričku promenljivu godina u kategorijalnu, što modelima mašinskog učenja olakšava prepoznavanje nelinearnih šablona po generacijama.

Hide

```
dataset_clean$customer_age_group = cut(
  dataset_clean$customer_age,
  breaks = c(0, 25, 40, 60, 100),
  labels = c("Mladi (do 25)", "Srednje doba (26-40)", "Stariji (41-60)", "Veterani (60+)"),
  include.lowest = TRUE
)
```

4.5 Tržišne i digitalne karakteristike (Market & Digital Features)

price_advantage: Prikazuje našu cenovnu prednost u odnosu na konkurenciju, dobijenu preko indeksa cene konkurencije (competitor_price_index).

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
dataset_clean$price_advantage = ifelse(
  !is.na(dataset_clean$competitor_price_index),
  (1-dataset_clean$competitor_price_index ) * 100,
  NA
)
```

effective_discount: Kombinuje procenat odobrenog popusta i broj aktivnih promocija da vidimo ukupni efekat popusta na kupca.

Hide

```
dataset_clean$effective_discount = dataset_clean$discount_percentage * dataset_clean$num_promotion
s
```

estimated_conversions: Daje procenu broja ostvarenih kupovina na osnovu poseta sajtu i stope konverzije.

Hide

```
dataset_clean$estimated_conversions = dataset_clean$website_traffic * dataset_clean$conversion_rat
e
```

traffic_per_follower: Meri odnos poseta sajtu i broja pratilaca na društvenim mrežama.

Hide

```
dataset_clean$traffic_per_follower = dataset_clean$website_traffic / (dataset_clean$social_media_f
ollowers + 1)
```

4.6 Interakcije feature-a (Feature Interactions)

segment_product: Kombinuje segment kupaca i kategoriju proizvoda, da vidimo koji tipovi kupaca preferiraju koje proizvode.

Hide

```
dataset_clean$segment_product = as.factor(paste(dataset_clean$customer_segment, dataset_clean$prod
uct_category, sep = "_"))
```

season_product: Spaja sezonu i kategoriju proizvoda radi praćenja sezonskih promena u potražnji.

Hide

```
dataset_clean$season_product = as.factor(paste(dataset_clean$season, dataset_clean$product_categor
y, sep = "_"))
```

channel_segment: Povezuje kanal prodaje sa segmentom kupaca, otkrivajući preko kog kanala koji kupci najčešće trguju.

Hide

```
dataset_clean$channel_segment = as.factor(paste(dataset_clean$sales_channel, dataset_clean$custome
r_segment, sep = "_"))
```

4.7 Provera i verifikacija novih feature-a

Proveravamo sve nove feature kako bismo utvrdili osnovnu statistiku, potvrdili ispravnost formula i osigurali da u njima nema neočekivanih anomalija.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
engineered_cols = c(
  "total_ad_spend", "online_share", "budget_utilization", "unspent_budget",
  "spend_per_rep", "spend_per_promotion", "budget_per_traffic", "log_budget",
  "log_traffic", "log_followers", "purchase_frequency", "price_advantage",
  "effective_discount", "estimated_conversions", "traffic_per_follower"
)

summary(dataset_clean[, engineered_cols])
colSums(is.na(dataset_clean[, engineered_cols]))
sapply(dataset_clean[, engineered_cols], function(x) sum(is.infinite(x)))
```

Nakon detaljnog pregleda statistike, potvrđeno je da su svi novi feature-i uspešno generisana i da se kreću u očekivanim i logičnim opsezima.

4.8 Prevencija curenja podataka (Target Leakage Prevention)

Treba programski osigurati i verifikovati da nijedan od novih feature-a u svom nazivu ili definiciji ne koristi našu ciljnu promenljivu (sales_revenue_usd), čime sprečavamo direktno curenje podataka.

Hide

```
target_variable = "sales_revenue_usd"
contains_target = any(grepl(target_variable, engineered_cols, fixed = TRUE))
stopifnot("GREŠKA: Curenje podataka! Novi feature-i koriste sales_revenue_usd!" = !contains_targe
t)
print("Verifikacija uspešna: Nijedan kreirani feature ne koristi ciljnu promenljivu (sales_revenue
```

Uspešno smo proverili da ni jedan novi feature ne koristi ciljnu promenljivu sales_revenue_usd

U ovom koraku treba da analiziramo korelacije između novih numeričkih feature-a i ciljne promenljive (sales_revenue_usd) kako bismo otkrili da li neki od feature-a ima nerealno visoku ili sumnjivu povezanost koja bi ukazivala na indirektno curenje podataka (target leakage).

Hide

```
num_engineered = c(
  "total_ad_spend", "online_share", "budget_utilization", "unspent_budget",
  "spend_per_rep", "spend_per_promotion", "budget_per_traffic", "log_budget",
  "log_traffic", "log_followers", "purchase_frequency", "price_advantage",
  "effective_discount", "estimated_conversions", "traffic_per_follower"
)
target_correlations = cor(dataset_clean[, num_engineered], dataset_clean[[target_variable]], use =
"complete.obs")
print(target_correlations)
```

Pregledom matrice korelacija vidimo da se sve vrednosti kreću u umerenim i očekivanim granicama.Nema sumnjivih korelacija bliskih 1.0, što potvrđuje da nijedan novokreirani atribut ne odaje informaciju o krajnjem prihodu unapred i da nema opasnosti od target leakage-a. Time smo i zvanično dokumentovali da su sve provere i mere zaštite od curenja podataka uspešno sprovedene.

5 Eksplorativna analiza podataka (EDA) i analize outlier-a

Loading [MathJax]/jax/output/HTML-CSS/jax.js

U ovom poglavlju sprovodimo univarijatnu analizu numeričkih i kategorijskih feature-a, analiziramo distribuciju i transformaciju ciljne promenljive (sales_revenue_usd), kao i detaljnu analizu outlier-a primenom IQR metode.

5.1 Univarijatna analiza numeričkih i kategorijskih feature-a

Univarijatna analiza nam omogućava da sagledamo distribuciju, oblike asimetrije i zastupljenost vrednosti kroz ključne numeričke i kategorijske atribute.

5.1.1 Analiza marketing budžeta (marketing_budget_usd)

Hide

```
ggplot(dataset_clean, aes(x = marketing_budget_usd)) +
  geom_histogram(bins = 30, fill = "#3498DB", color = "#2C3E50", alpha = 0.8) +
  theme_minimal() +
  labs(title = "Histogram: Marketing Budžet",
       x = "Marketing Budžet (USD)",
       y = "Frekvencija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Histogram marketing budžeta ukazuje na to da su iznosi većinski locirani u nižem i srednjem opsegu, dok se frekvencija postepeno smanjuje ka većim vrednostima. Ova desna asimetrija ukazuje na to da je većina kampanja imala umerena ulaganja, uz manji broj kampanja sa visokim budžetima.

5.1.2 Analiza poseta sajtu (website_traffic)

Hide

```
ggplot(dataset_clean, aes(x = website_traffic)) +
  geom_histogram(bins = 30, fill = "#E67E22", color = "#2C3E50", alpha = 0.8) +
  theme_minimal() +
  labs(title = "Histogram: Posete sajtu",
       x = "Website Traffic",
       y = "Frekvencija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Histogram analiza poseta sajtu ukazuje na to da većina opservacija beleži niži nivo posjećenosti, dok mali broj objava ili kampanja privlači izuzetno veliki saobraćaj, što stvara jaku pozitivnu asimetriju.

5.1.3 Analiza broja pratilaca na društvenim mrežama (social_media_followers)

Hide

```
ggplot(dataset_clean, aes(x = social_media_followers)) +
  geom_histogram(bins = 30, fill = "#9B59B6", color = "#2C3E50", alpha = 0.8) +
  theme_minimal() +
  labs(title = "Histogram: Pratioci na društvenim mrežama",
       x = "Broj pratilaca",
       y = "Frekvencija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Histogram pratioci na društvenim mrežama ukazuje na to da broj pratilaca pokazuje visoku koncentraciju vrednosti na početnom delu skale. I ovde je primetno odsustvo normalne raspodele, što opravdava ranije uvedenu logaritamsku transformaciju radi lakšeg modelovanja.

5.1.4 Transakcije po regionima (region)

Hide

```
ggplot(dataset_clean, aes(x = region, fill = region)) +
  geom_bar( alpha = 0.8, show.legend = TRUE) +
  theme_minimal() +
  labs(title = "Bar plot: Zastupljenost po regionima",
       x = "Region",
       y = "Broj transakcija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

Grafik pokazuje značajne varijacije u ukupnom broju transakcija po regionima. Dok gradovi poput Kaira, Rijada i Aleksandrije beleže najveći obim transakcija, drugi regioni poput Amana, Kasablanke i Kuvajta imaju znatno manju zastupljenost u bazi.

5.1.5 Kategorije proizvoda (product_category)

Hide

```
ggplot(dataset_clean, aes(x = product_category, fill = product_category)) +
  geom_bar(alpha = 0.8, show.legend = TRUE) +
  theme_minimal() +
  labs(title = "Bar plot: Kategorije proizvoda",
       x = "Kategorija proizvoda",
       y = "Broj transakcija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_text(angle = 30, hjust = 1))
```

Kategorije proizvoda su ravnomerno zastupljene u uzorku. Nema ekstremnih oscilacija niti dominacije jedne kategorije nauštrb ostalih, što obezbeđuje kvalitetnu osnovu i stabilnost podataka za dalje modelovanje i analizu uticaja pojedinačnih kategorija na prihode.

5.2 Analiza i transformacija ciljne promenljive (sales_revenue_usd)

5.2.1 Izračunavanje asimetrije originalne ciljne promenljive

Hide

```
calculate_skewness = function(x) {
  n = length(x)
  sum((x - mean(x))^3) / (n * sd(x)^3)
}

original_skew = calculate_skewness(dataset_clean$sales_revenue_usd)
print(paste("Asimetrija (Skewness) originalnog prihoda:", round(original_skew, 4)))
```

To je izuzetno visoka pozitivna asimetrija, što znači da je prihod narušen malim brojem ogromnih vrednosti koje vuku prosek daleko udesno.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

5.2.2 Poređenje originalne i logaritamske distribucije prihoda

Hide

```
dataset_clean$log_revenue = log(dataset_clean$sales_revenue_usd)
revenue_comparison_df = dataset_clean %>%
  select(sales_revenue_usd, log_revenue) %>%
  pivot_longer(everything(), names_to = "Metric", values_to = "Value")

ggplot(revenue_comparison_df, aes(x = Value, fill = Metric)) +
  geom_histogram(bins = 30, color = "#2C3E50", alpha = 0.8, show.legend = FALSE) +
  facet_wrap(~ Metric, scales = "free", ncol = 2,
             labeller = labeller(Metric = c("sales_revenue_usd" = "Originalni prihodi (USD)",
                                            "log_revenue" = "Logaritmovani prihodi (Log Revenu
e)"))) +
  theme_minimal() +
  labs(title = "Poređenje distribucije prihoda pre i posle log-transformacije",
       x = "Vrednost feature-a", y = "Frekvencija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        strip.text = element_text(face = "bold", size = 11))
```

Matematička analiza potvrđuje izuzetno visoku pozitivnu asimetriju originalnog prihoda (skewness = 6.63). Sa desne strane grafikona prikazan je originalni prihod sa izraženim dugim desnim repom. Primena logaritamske transformacije (levo) uspešno svodi distribuciju na stabilan, približno normalan oblik

5.2.3 Q-Q plot za proveru normalnosti log-transformisanog prihoda

Hide

```
set.seed(123)

sample_revenue = sample(dataset_clean$log_revenue, min(5000, length(dataset_clean$log_revenue)))

qqnorm(sample_revenue, main = "Q-Q Plot - Log-transformisani prihod", col = "#2980B9")
qqline(sample_revenue, col = "#C0392B", lwd = 2)
```

Q-Q plot nam pokazuje da se tačkice u sredini lepo drže linije, uz neka manja odstupanja na krajevima, ovo nam potvrđuje da je log-transformacija odlično odradila posao i dovela podatke u formu koja je sasvim stabilna za dalje modelovanje.

5.3 Analiza outlier-a

Identifikujemo ekstremne opservacije za prihod primenom interkvartilnog opsega (IQR). Važna napomena: outlier-i se ne brišu automatski.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
Q1 = quantile(dataset_clean$sales_revenue_usd, 0.25)
Q3 = quantile(dataset_clean$sales_revenue_usd, 0.75)
IQR_value = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR_value
upper_bound = Q3 + 1.5 * IQR_value

outliers_revenue = dataset_clean$sales_revenue_usd[dataset_clean$sales_revenue_usd < lower_bound |
dataset_clean$sales_revenue_usd > upper_bound]

print(paste("Donja granica:", round(lower_bound, 2)))
print(paste("Gornja granica:", round(upper_bound, 2)))
print(paste("Broj identifikovanih outlier-a po IQR metodi:", length(outliers_revenue)))
print(paste("Procenat outlier-a u ukupnom skupu:", round((length(outliers_revenue) / nrow(dataset_
clean)) * 100, 2), "%"))
```

Formula nam je izbacila gornju granicu na 13.612,81 USD, a sve preko toga je označila kao autlajer. Tako smo dobili 3.816 takvih redova, što mu dođe oko 6.36% celog skupa.

Iako deluje kao veliki procenat, u svetu prodaje i marketinga skroz je normalno da neke kampanje iskoče i donesu mnogo veće pare od proseka. To nisu nikakve greške, već naši najuspešniji rezultati. Upravo zato ne diramo i ne brišemo ove outliere — da ih sklonimo, osiromašili bismo podatke i model ne bi imao na čemu da uči šta zapravo donosi najveći profit.

5.4 Korelaciona analiza feature-a

U ovom delu sprovodimo ispitivanje međusobne povezanosti numeričkih atributa. Pošto pojedine promenljive u našem skupu podataka pokazuju izraženu asimetriju i prisustvo ekstremnih vrednosti, koristimo Spearman-ov koeficijent korelacije, pošto on meri monotonu povezanost i otporniji je na outlier-e u odnosu na standardni Pearson-ov koeficijent.

5.4.1 Izračunavanje Spearmanove matrice korelacije

Hide

```
numeric_cols_for_corr = dataset_clean %>%
  select(
    sales_revenue_usd, marketing_budget_usd, ad_spend_online_usd,
    ad_spend_offline_usd, num_promotions, discount_percentage,
    num_sales_representatives, customer_age, customer_satisfaction_score,
    competitor_price_index, website_traffic, conversion_rate,
    email_open_rate, social_media_followers, days_since_last_purchase,
    num_previous_purchases, total_ad_spend, online_share,
    budget_utilization, unspent_budget, spend_per_rep,
    spend_per_promotion, budget_per_traffic, purchase_frequency,
    traffic_per_follower
  )


spearman_matrix = cor(numeric_cols_for_corr, method = "spearman", use = "complete.obs")

revenue_correlations = sort(spearman_matrix["sales_revenue_usd", ], decreasing = TRUE)
print("Spearman-ove korelacije sa ciljnom promenljivom (sales_revenue_usd):")
print(round(revenue_correlations, 3))
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Kada izdvojimo korelacije sa ciljnom promenljivom (sales_revenue_usd), vidimo da finansijska ulaganja (marketing_budget_usd, total_ad_spend) ostvaruju umerene pozitivne vrednosti (oko 0.5), dok feature poput poseta sajtu (website_traffic) i društvenih mreža pokazuju potpuno neutralnu korelaciju. To ukazuje na to da veći budžeti prate veći prihod, ali da sama masovna vidljivost ne garantuje profit bez efikasnih konverzija.

5.4.2 Vizuelni prikaz korelacije (Correlation matrix plot)

Umesto klasične velike matrice koja može biti nepregledna sa velikim brojem feature-a, korelacije direktno u odnosu na našu ciljnu promenljivom (sales_revenue_usd) prikazujemo preglednim horizontalnim stubastim dijagramom.

Hide

```
revenue_corr_df = data.frame(
  Feature = names(revenue_correlations),
  Correlation = as.numeric(revenue_correlations)
) %>%
  filter(Feature != "sales_revenue_usd") # Uklanjamo samopovezivanje (1.0)

ggplot(revenue_corr_df, aes(x = reorder(Feature, Correlation), y = Correlation, fill = Correlation
> 0)) +
  geom_col(alpha = 0.85, color = "#2C3E50") +
  coord_flip() +
  scale_fill_manual(values = c("#E74C3C", "#3498DB"), guide = "none") +
  theme_minimal() +
  labs(title = "Spearman-ova korelacija feature-a sa prihodom (sales_revenue_usd)",
       x = "Obeležje",
       y = "Koeficijent korelacije") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
        axis.text.y = element_text(size = 9))
```

Sa grafika se jasno izdvajaju tri grupe feature-a u odnosu na prihod (sales_revenue_usd):

Snažne pozitivne veze: Finansijska ulaganja (marketing_budget_usd, total_ad_spend, onlajn/oflajn budžeti) imaju najveći uticaj sa korelacijama oko 0.5.

Neutralna zona: Većina ostalih feature-a (poput poseta sajtu i ocena zadovoljstva) ima korelaciju blisku nuli, što znači da nemaju direktan linearni uticaj na prihod.

Blage negativne veze: Indeks cena konkurencije i popusti (discount_percentage) nalaze se na dnu tabele sa blago negativnim koeficijentom korelacije, što ukazuje na to da visina odobrenog popusta u ovom skupu podataka nema direktnu pozitivnu povezanost sa ukupnim ostvarenim prihodom.

5.4.3 Identifikovanje snažnih veza i redundantnih feature-a

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
get_high_correlations = function(matrix, threshold = 0.75) {
  cor_df = as.data.frame(as.table(matrix))
  names(cor_df) = c("Var1", "Var2", "Correlation")
  cor_df = cor_df %>%
    filter(as.character(Var1) < as.character(Var2)) %>%
    filter(abs(Correlation) >= threshold) %>%
    arrange(desc(abs(Correlation)))
  return(cor_df)
}

high_corr_pairs = get_high_correlations(spearman_matrix, threshold = 0.75)
print(head(high_corr_pairs, 10))
```

Analizom parova izdvaja se izražena multikolinearnost među finansijskim obeležjima:

Ekstremna redundantnost budžeta: Promenljive total_ad_spend, marketing_budget_usd, kao i onlajn i oflajn budžeti pokazuju korelacije između 0.91 i 0.99, što je logično jer su međusobno izvedene i zbirno povezane. Visoku povezanost beleži i neiskorišćeni budžet (unspent_budget).

Implikacija za modelovanje: Zbog visoke multikolinearnosti, u kasnijoj fazi izgradnje modela (poput linearne regresije) biće neophodno ukloniti redundantne promenljive kako bi se sprečila nestabilnost koeficijenata.

5.5 Kategorijska analiza

U ovom delu analiziramo uticaj i razlike ciljne promenljive (`sales_revenue_usd`) u zavisnosti od ključnih kategorijskih feature-a (kao što su region prodaje i kategorija proizvoda). Koristimo i vizuelne metode (boxplote).

5.5.1 Prihod po kategorijama proizvoda

Hide

```
ggplot(dataset_clean, aes(x = product_category, y = sales_revenue_usd, fill = product_category)) +
  geom_boxplot(alpha = 0.8, outlier.color = "#C0392B", outlier.size = 1.5) +
  theme_minimal() +
  labs(title = "Boxplot: Ostvareni prihod po kategorijama proizvoda",
       x = "Kategorija proizvoda",
       y = "Prihod (USD)") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
        axis.text.x = element_text(angle = 30, hjust = 1),
        legend.position = "none")
```

Boxplot dijagram pokazuju ujednačene medijane i interkvartilne opsege kroz sve kategorije proizvoda, uz prisustvo pojedinačnih autlajera sa višim prihodima, što ukazuje na ravnomerno ostvarene rezultate bez izražene dominacije pojedine grupe.

5.5.2 Prihod po regionima prodaje

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
ggplot(dataset_clean, aes(x = region, y = sales_revenue_usd, fill = region)) +
  geom_boxplot(alpha = 0.8, outlier.color = "#C0392B", outlier.size = 1.5) +
  theme_minimal() +
  labs(title = "Boxplot: Ostvareni prihod po regionima prodaje",
       x = "Region prodaje",
       y = "Prihod (USD)") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
        axis.text.x = element_text(angle = 30, hjust = 1),
        legend.position = "none")
```

Boxplot dijagram pokazuju ujednačene medijane i interkvartilne opsege po regionima prodaje, uz prisustvo pojedinačnih outlier sa natprosečnim vrednostima, što ukazuje na ravnomerno ostvarene prihode na svim posmatranim tržištima.

5.6 Vremenska analiza i sezonski obrasci

U ovom delu analiziramo kretanje ostvarenog prihoda kroz vreme (po godinama, kvartalima i mesecima) koristeći već pripremljene vremenske atribute u skupu podataka, kako bismo identifikovali trendove i uticaj Q4 efekta.

5.6.1 Agregacija prihoda po godinama

Hide

```
#1. Prihod po godinama (ekstrakcija godine iz 'date' kolone)
revenue_by_year=dataset_clean %>%
  mutate(year =(year(as.Date(date)))) %>%
  group_by(year) %>%
  summarise(
    total_revenue = sum(sales_revenue_usd, na.rm = TRUE),
    mean_revenue = mean(sales_revenue_usd, na.rm = TRUE),
    n_transactions = n()
  )
print(revenue_by_year)
```

Poslovanje je izuzetno stabilno od 2020. do 2023. godine, uz ujednačen ukupan prihod (86,6–89,9 miliona USD) i stabilan obim od oko 15.000 transakcija godišnje.

5.6.2 Agregacija prihoda po kvartalima

Hide

```
# 2. Prihod po kvartalima (korišćenje postojeće kolone 'quarter')

revenue_by_quarter=dataset_clean %>%
  mutate(quarter =(quarter(as.Date(date)))) %>%
  group_by(quarter) %>%
  summarise(
    total_revenue = sum(sales_revenue_usd, na.rm = TRUE),
    mean_revenue = mean(sales_revenue_usd, na.rm = TRUE),
    n_transactions = n()
  )
print(revenue_by_quarter)
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Prihodi rastu kroz kvartale i dostižu maksimum od 113,4 miliona USD u četvrtom kvartalu (Q4), čime se potvrđuje snažan sezonski uticaj kraja godine.

5.6.3 Agregacija prihoda po mesecima

Hide

```
# 3. Prihod po mesecima (korišćenje postojeće kolone 'month')
revenue_by_month=dataset_clean %>%
  mutate(month = month(as.Date(date))) %>%
  group_by(month) %>%
  summarise(
    total_revenue = sum(sales_revenue_usd, na.rm = TRUE),
    mean_revenue = mean(sales_revenue_usd, na.rm = TRUE),
    n_transactions = n()
  )

print(revenue_by_month)
```

Promet je stabilan u prvoj polovini godine (22–27 miliona USD mesečno), dok u oktobru, novembru i decembru naglo skače na preko 36 miliona USD mesečno.

5.6.4 Vizuelizacija sezonskih obrazaca i istraživanje Q4 efekta

Hide

```
ggplot(revenue_by_month, aes(x = factor(month), y = total_revenue, group = 1)) +
  geom_line(color = "#2980B9", linewidth = 1.2) +
  geom_point(color = "#C0392B", size = 3) +
  scale_x_discrete(labels = c("Jan", "Feb", "Mar", "Apr", "Maj", "Jun",
                              "Jul", "Avg", "Sep", "Okt", "Nov", "Dec")) +
  scale_y_continuous(labels = label_number(scale = 1e-6, suffix = "M")) +
  theme_minimal() +
  labs(
    title = "Sezonski obrasci: Ukupan prihod po mesecima",
    x = "Mesec",
    y = "Ukupan prihod (u milionima USD)"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
    axis.text.x = element_text(angle = 0, hjust = 0.5)
  )
```

Dijagram jasno prikazuje sezonske fluktuacije sa strmim usponom u četvrtom kvartalu (sa kulminacijom u oktobru). Ova ujednačenost i pravilnost kroz godine ukazuju na to da podaci imaju sintetička obeležja, što predstavlja jedno od metodoloških ograničenja pri interpretaciji u realnom poslovnom okruženju.

6 Feature Selection

6.1 Multikolinearnost

U ovom poglavlju prelazimo na analizu multikolinearnosti i definisanje optimalnog skupa feature-a. Primenom faktora inflacije varijanse (VIF - Variance Inflation Factor) identifikujemo redundantne promenljive koje mogu da naruše stabilnost i interpretabilnost naših regresionih modela.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

6.1.1 Izračunavanje VIF vrednosti

Hide

```
vif_model = lm(log_revenue ~log_budget+ total_ad_spend+spend_per_rep+spend_per_promotion+estimated
_conversions+
                 num_promotions + discount_percentage + num_sales_representatives +
                 customer_age + customer_satisfaction_score + competitor_price_index +
                 website_traffic + conversion_rate + email_open_rate +
                 social_media_followers + days_since_last_purchase + num_previous_purchases +
                 online_share + budget_utilization + purchase_frequency +
                 region + product_category + season + customer_segment + sales_channel,
               data = dataset_clean)

# Računanje VIF-a (koristimo generalizovani VIF - GVIF koji ispravno tretira i kategorijske promen
ljive sa više nivoa)
vif_values = vif(vif_model)
print("--- GVIF vrednosti za sva feature-a  ---")
print(round(vif_values, 2))
```

Analizom dobijenih VIF vrednosti uočavamo da sve promenljive imaju vrednosti daleko ispod praga od 5 (maksimalne su estimated_conversions sa 3.48 i total_ad_spend sa 3.41). Ostale promenljive (uključujući log_budget sa VIF = 1.87) takođe nemaju problem sa kolinearnošću, čime je multikolinearnost svedena na prihvatljiv nivo za stabilno procenjivanje modela.

Hide

```
gvif_column = vif_values[, "GVIF"]
selected_features = names(gvif_column[gvif_column <5])
print("Konačna lista stabilnih feature-a:")
print(selected_features)
```

Nakon provere zadržali smo sve feature koji imaju prag manji od 5

6.2 Statistička selekcija feature-a

Nakon rešavanja problema multikolinearnosti, prelazimo na statističku selekciju feature-a kako bismo kvantifikovali linearne i nelinearne veze sa ciljnom promenljivom (log_revenue), izdvojili najuticajnije prediktore i eliminisali slabe ili irelevantne atribute koji ne doprinose prediktivnoj moći modela.

6.2.1 Rangiranje numeričkih feature-a po Spearman-ovoj korelaciji

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
numeric_features = selected_features[sapply(dataset_clean[selected_features], is.numeric)]


spearman_corrs = sapply(numeric_features, function(feat) {

  if(sd(dataset_clean[[feat]], na.rm = TRUE) == 0) {
    return(NA)
  }
  cor(dataset_clean[[feat]], dataset_clean$log_revenue, method = "spearman", use = "complete.obs")
})


spearman_ranking = data.frame(
  Feature = names(spearman_corrs),
  Spearman_Rho = round(spearman_corrs, 4),
  Abs_Correlation = round(abs(spearman_corrs), 4)
)

spearman_ranking = na.omit(spearman_ranking)


spearman_ranking = spearman_ranking[order(-spearman_ranking$Abs_Correlation), ]
rownames(spearman_ranking) = NULL

print(spearman_ranking)
```

Analizom dobijenih vrednosti Spearman-ove korelacije uočavamo da vodeća mesta zauzimaju budžetski prediktori poput log_budget,total_ad_spend,spend_per_rep i spend_per_promotion. Sa druge strane, ostale numeričke promenljive (uključujući num_previous_purchases i conversion_rate) pokazuju znatno slabiju ili gotovo zanemarljivu pojedinačnu povezanost sa ciljnom promenljivom.

6.2.2 Rangiranje kategorijskih feature-a po veličini efekta

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
categorical_features = selected_features[sapply(dataset_clean[selected_features], function(x) is.f
actor(x) || is.character(x))]

calculate_eta_sq = function(feat, target, data) {
  aov_model = aov(data[[target]] ~ as.factor(data[[feat]]))
  summary_aov = summary(aov_model)[[1]]
  ss_effect = summary_aov["Sum Sq"][1, 1]
  ss_total = sum(summary_aov["Sum Sq"])
  return(ss_effect / ss_total)
}

effect_sizes = sapply(categorical_features, function(feat) {
  calculate_eta_sq(feat, "log_revenue", dataset_clean)
})

categorical_ranking = data.frame(
  Feature = names(effect_sizes),
  Eta_Squared = as.numeric(round(effect_sizes, 4))
)

categorical_ranking = categorical_ranking[order(-categorical_ranking$Eta_Squared), ]

knitr::kable(categorical_ranking, row.names = FALSE)
```

Analizom možemo da vidimo da kategorijske promenljive poput customer_segment i product_category snažno utiču na varijaciju prihoda. Sa druge strane, promenljiva region pokazuje zanemarljiv uticaj (blizu nule), što ukazuje na to da ne doprinosi prediktivnoj moći modela.

6.2.3 Identifikovanje slabih prediktora

Izdvajamo slabe numeričke i kategorijske prediktore koji ne nose značajnu informaciju za model.

Hide

```
weak_numeric = spearman_ranking$Feature[spearman_ranking$Abs_Correlation < 0.05]

print("Identifikovani slabi numerički prediktori:")
print(weak_numeric)
```

Hide

```
weak_categorical = categorical_ranking$Feature[categorical_ranking$Eta_Squared < 0.01]
print("Identifikovani slabi kategorijski prediktori:")
print(weak_categorical)
```

Hide

```
final_features = setdiff(selected_features, c(weak_numeric, weak_categorical))
print("Konačna lista feature-a nakon statističke selekcije:")
print(final_features)
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Primenom definisanih statističkih pragova uspešno smo izdvojili atribute čija je prediktivna snaga zanemarljiva. Njihovim uklanjanjem iz skupa selected_features dobijamo konačnu listu (final_features) koja sadrži isključivo relevantne promenljive sa dokazanim linearnim ili grupnim uticajem, čime smanjujemo šum u podacima i sprečavamo rizik od overfitting-a.

6.2.4 Poređenje modela: Full vs. Selected Feature Set

Kako bismo opravdali redukciju feature-a i dokazali da odbacivanje slabih i redundantnih prediktora nije narušilo kvalitet modela, uporedićemo performanse linearnog modela sa svim obeležjima (Full Model) i modela sa konačno izabranim skupom feature-a (Selected Model)

Hide

```
exclude_cols = c("log_revenue", "sales_revenue_usd", "id", "date")
full_predictors = setdiff(names(dataset_clean), exclude_cols)


formula_full = as.formula(paste("log_revenue ~", paste(full_predictors, collapse = " + ")))
model_full = lm(formula_full, data = dataset_clean)

formula_selected = as.formula(paste("log_revenue ~", paste(final_features, collapse = " + ")))
model_selected = lm(formula_selected, data = dataset_clean)


comparison_summary = data.frame(
  Model = c("Full Model (Svi opravdani feature-i)", "Selected Model (Redukovan skup)"),
  Number_of_Predictors = c(length(full_predictors), length(final_features)),
  Adjusted_R2 = c(summary(model_full)$adj.r.squared, summary(model_selected)$adj.r.squared),
  AIC = c(AIC(model_full), AIC(model_selected))
)
print(comparison_summary)
```

Poređenje modela ukazuje na odličan kompromis: Full Model sa 46 feature-a postiže R^2_adj = 0.943, ali je preopterećen multikolinearnošću i prevelikom kompleksnošću. Sa druge strane, Selected Model sa svega 13 feature-a zadržava visok R^2_adj = 0.902 (objašnjava preko 90% varijanse), pri čemu su uspešno sačuvani ključni budžetski prediktori. Blagi pad objašnjene varijanse od oko 4% predstavlja minimalnu cenu za drastično smanjenje kompleksnosti, potpunu eliminaciju multikolinearnosti i dobijanje stabilnog i interpretabilnog modela.

6.3 Model-based selekcija feature-a

Nakon statističkog filtriranja, primenjujemo napredne pristupe zasnovane na mašinskom učenju kako bismo dodatno potvrdili značaj feature-a i eliminisali preostali šum. Koristimo Random Forest (za procenu nelinearne važnosti promenljivih preko smanjenja srednje kvadratne greške) i Lasso regularizaciju (L1 penalizaciju koja automatski redukuje koeficijente slabih prediktora na nulu).

6.3.1 Random Forest – Rangiranje važnosti feature-a

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
set.seed(123)

rf_model = randomForest(log_revenue ~ .,
                        data = dataset_clean[, c(final_features, "log_revenue")],
                        ntree = 150,
                        importance = TRUE)

rf_imp = importance(rf_model, type = 1)
rf_ranking = data.frame(
  Feature = rownames(rf_imp),
  Importance = round(rf_imp[, 1], 4)
)

rf_ranking = rf_ranking[order(-rf_ranking$Importance), ]
rownames(rf_ranking) = NULL


print(head(rf_ranking, 10))
```

Random Forest analiza pokazuje koji feature-i imaju najveću prediktivnu moć pri obradi složenih, nelinearnih interakcija. Vodeće pozicije ubedljivo zauzimaju kategorijski atributi, pri čemu segment kupaca (customer_segment) i kategorija proizvoda (product_category). Među numeričkim prediktorima, najveći nelinearni doprinos pokazuju istorija kupovina (num_previous_purchases), stopa konverzije (conversion_rate) i zadovoljstvo kupaca (customer_satisfaction_score), dok budžetske varijable (log_budget i total_ad_spend) dodatno stabilizuju model u top 10 prediktora.

6.3.2 Lasso regularizacija (L1 penalizacija)

Hide

```
x_mat = model.matrix(log_revenue ~ ., data = dataset_clean[, c(final_features, "log_revenue")])[,
-1]
y_vec = dataset_clean$log_revenue


set.seed(42)
cv_lasso = cv.glmnet(x_mat, y_vec, alpha = 1, nfolds = 5)
best_lambda = cv_lasso$lambda.min


lasso_model = glmnet(x_mat, y_vec, alpha = 1, lambda = best_lambda)
lasso_coefs = coef(lasso_model)


lasso_selected_dummies = rownames(lasso_coefs)[lasso_coefs[, 1] != 0]
lasso_selected_dummies = setdiff(lasso_selected_dummies, "(Intercept)")

print(paste("Broj selektovanih feature-a nakon Lasso regularizacije je :",length(lasso_selected_du
```

Lasso regularizacija je zadržala svih 23 dummy prediktora (nijedan koeficijent nije sveden na nulu), što potvrđuje da su sve izabrane varijable značajne.

6.3.3 Poređenje selektovanih feature-a i kreiranje konačne liste

Loading [MathJax]/jax/output/HTML-CSS/jax.js

U cilju postizanja maksimalne robusnosti, uporedićemo feature dobijeni statističkim metodama, Random Forest-om i Lasso regularizacijom, te formirati konačni skup feature-a za finalno modelovanje.

Hide

```
rf_top_features = rf_ranking$Feature[rf_ranking$Importance > 10]

lasso_original_features = unique(sapply(lasso_selected_dummies, function(col) {
  matched = final_features[sapply(final_features, function(f) grepl(f, col))]
  if(length(matched) > 0) return(matched[1]) else return(col)
}))


final_model_features = intersect(intersect(final_features, rf_top_features), lasso_original_featur
es)


if(length(final_model_features) == 0) {
  final_model_features = intersect(final_features, rf_top_features)
}

print("Konačna lista feature-a za mašinsko učenje (Final Feature List):")
print(final_model_features)
print(paste("Ukupan broj feature-a u konačnom skupu:", length(final_model_features)))
```

Kombinovanjem statističkih metoda, Random Forest-a i Lasso regularizacije formiran je optimalan skup od 13 feature-a (9 numeričkih i 4 kategorijska). Ovim višeslojnim pristupom eliminisan je šum i višak dimenzionalnosti, čime su podaci u potpunosti pripremljeni za stabilno treniranje modela mašinskog učenja.

7 Machine learning

7.1 Priprema skupa podataka za modelovanje

Za trening modela koristimo konačno izabrane prediktore iz prethodnog koraka (final_model_features), pridružujući im ciljnu promenljivu log_revenue, kao i originalnu sales_revenue_usd radi kasnijeg poređenja metrika.

Hide

```
ml_features_data = dataset_clean[, c(final_model_features, "log_revenue", "sales_revenue_usd")]

print(paste("Ukupan broj feature-a pripremljenih za modelovanje:", length(final_model_features)))
print(paste("Dimenzije matrice za mašinsko učenje:", nrow(ml_features_data), "redova i", ncol(ml_f
eatures_data), "kolona"))
```

7.2 Kreiranje Train/Test split-a

Primenjujemo standardnu metodologiju podele podataka gde se 80% opservacija koristi za trening i učenje modela, 20% ukupnog skupa ostaje izdvojeno u test set.

7.2.1 Postavljanje seed-a i indeksiranje

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
set.seed(123)

train_sample_index = sample(1:nrow(ml_features_data), size = 0.8 * nrow(ml_features_data))


train_set = ml_features_data[train_sample_index, ]
test_set  = ml_features_data[-train_sample_index, ]

print(paste("Broj instanci u trening setu (80%):", nrow(train_set)))
print(paste("Broj instanci u test setu (20%):", nrow(test_set)))
```

7.2.2 Provera distribucije ciljne promenljive u Train i Test setu

Kako bismo osigurali da podela nije promenila strukturu uzorka, upoređujemo osnovne statističke pokazatelje ciljne promenljive log_revenue u oba skupa.

Hide

```
train_target_summary = summary(train_set$log_revenue)
test_target_summary  = summary(test_set$log_revenue)

print("--- Statistika log_revenue u Trening setu ---")
print(train_target_summary)
```

Hide

```
print("--- Statistika log_revenue u Test setu ---")
print(test_target_summary)
```

Statistički pokazatelji (srednje vrednosti, medijane i kvartili) praktično su identični u trening i test setu, što potvrđuje da je nasumična podela uspešno sačuvala strukturu ciljne promenljive.

7.2.3 Vizuelno poređenje distribucija

Hide

```
split_distribution_df = bind_rows(
  mutate(train_set, Split_Group = "Train Set (80%)"),
  mutate(test_set,  Split_Group = "Test Set (20%)")
)

ggplot(split_distribution_df, aes(x = log_revenue, fill = Split_Group)) +
  geom_density(alpha = 0.6, color = "#2C3E50") +
  theme_minimal() +
  labs(title = "Poređenje gustine raspodele log_revenue između Train i Test seta",
       x = "Log Revenue",
       y = "Gustina") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
        legend.title = element_blank())
```

Grafik gustine raspodele pokazuje potpuno preklapanje krivih za trening i test set, što i formalno potvrđuje ravnomernu i ispravnu izvršenu podelu podataka.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

7.3 Prevencija curenja podataka (Data Leakage Prevention)

Iako su imputacija i početna selekcija prediktora sprovedene nad celokupnim skupom podataka radi očuvanja statističke moći, treniranje i podešavanje hiperparametara svih ML modela rađeno je isključivo na trening setu (80%), dok je test set (20%) ostao potpuno izolovan za evaluaciju.

7.4 Preprocessing Pipeline - preprocesiranje podataka

U ovom poglavlju definisaćemo preprocesiranje podataka koji obuhvata kodiranje kategoričkih promenljivih, numeričku normalizaciju i strogu kontrolu transformacija kako bi se sprečilo curenje podataka (data leakage).

7.4.1 Definisanje pravila predobrade

Da bismo osigurali ispravnost modelovanja, poštujemo zlatno pravilo mašinskog učenja: svi parametri transformacije (srednje vrednosti, standardne devijacije, nivoi faktora i medijane) računaju se isključivo na trening setu (train_set), a zatim se preslikavaju na test set (test_set).

Hide

```
numeric_features = names(train_set)[sapply(train_set, is.numeric)]
numeric_features = setdiff(numeric_features, c("log_revenue", "sales_revenue_usd"))

categorical_features = names(train_set)[sapply(train_set, function(x) is.factor(x) || is.character
(x))]

print(paste("Broj numeričkih prediktora za obradu:", length(numeric_features)))
print(paste("Broj kategorijskih prediktora za kodiranje:", length(categorical_features)))
```

7.4.2 Dummy (factor) kodiranje kategorijskih promenljivih

Kategorijske promenljive transformišemo u set factor kolona. Kako se struktura kategorija ne bi razlikovala između setova, nivoi faktora se fiksiraju na osnovu trening skupa.

Hide

```
for(col in categorical_features) {
  train_set[[col]] = as.factor(train_set[[col]])
  test_set[[col]]  = factor(test_set[[col]], levels = levels(train_set[[col]]))
}
```

7.4.3 Numerička normalizacija

Za numeričke prediktore primenjujemo standardizaciju (z-score normalizaciju: oduzimanje srednje vrednosti trening seta i deljenje sa standardnom devijacijom trening seta).

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
# Izračunavanje srednje vrednosti i standardne devijacije ISKLJUČIVO na trening setu

train_means = sapply(train_set[, numeric_features, drop=FALSE], mean, na.rm = TRUE)
train_sds   = sapply(train_set[, numeric_features, drop=FALSE], sd, na.rm = TRUE)

# Funkcija za primenu skaliranja
scale_features = function(df, features, means, sds) {
  df_scaled = df
  for(col in features) {
    if(sds[col] > 0) {
      df_scaled[[col]] = (df[[col]] - means[col]) / sds[col]
    }
  }
  return(df_scaled)
}

train_set_processed = scale_features(train_set, numeric_features, train_means, train_sds)
test_set_processed  = scale_features(test_set, numeric_features, train_means, train_sds)

keep_cols = c(final_model_features, "log_revenue", "sales_revenue_usd")
train_set_processed = train_set_processed[, keep_cols]
test_set_processed  = test_set_processed[, keep_cols]
```

7.4.4 Završna verifikacija procesa predobrade

Hide

```
print(paste("Dimenzije obrađenog trening seta:", nrow(train_set_processed), "redova,", ncol(train_
set_processed), "kolona"))
print(paste("Dimenzije obrađenog test seta:", nrow(test_set_processed), "redova,", ncol(test_set_p
rocessed), "kolona"))
print("Preprocessing Pipeline je uspešno završen spreman za treniranje modela.")
```

7.5 Cross-validation - Unakrsna validacija

U ovom poglavlju definišemo proceduru unakrsne validacije (cross-validation) radi pouzdane procene performansi modela i zaštite od prevelikog prilagođavanja (overfitting).

7.5.1 Definisanje 5-fold unakrsne validacije i metrika

Skup podataka za trening se deli na 5 jednakih delova (foldova), gde se u svakoj iteraciji 4 dela koriste za treniranje, a preostali 1. deo za validaciju modela. Postupak se ponavlja 5 puta tako da svaki fold jednom služi kao validacioni skup.

Hide

```
# Konfiguracija 5-fold CV kontrole
train_control = trainControl(
  method = "cv",
  number = 5,
  savePredictions = "final",
  verboseIter = FALSE
)
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

7.5.2 Evidentiranje prosečnih vrednosti i standardne devijacije metrika

Tokom procesa unakrsne validacije, performanse se prate kroz više iteracija kako bi se zabeležila srednja vrednost (mean) i standardna devijacija (SD) za svaku izabranu metriku, što daje uvid u stabilnost modela.

Hide

```
cv_summary_table =data.frame(
  Parametar = c("Metod validacije", "Broj foldova (K)", "Evaluacione metrike", "Praćenje rezultat
a", "Čuvanje predikcija"),
  Vrednost = c("Cross-Validation (cv)", "5", "RMSE, MAE, R-squared", "Mean i Standardna devijacija
(SD)", "final")
)

knitr::kable(cv_summary_table, caption = "Konfigurisani parametri i metrike unakrsne validacije")
```

Ova tabela pregledno prikazuje konfigurisane parametre unakrsne validacije, navodeći primijenjene evaluacione metrike (RMSE, MAE, R^2) i način praćenja stabilnosti modela kroz srednje vrijednosti i standardne devijacije duž svih 5 foldova.

7.6 Baseline model (Referentni model)

Pre izgradnje naprednih modela mašinskog učenja, neophodno je formirati referentni (baseline) model koji služi kao donji prag uspešnosti i polazna tačka za poređenje.

7.6.1 Predikcija srednje vrednosti (Mean prediction)

Kao referencu koristimo prostju predikciju srednje vrednosti ciljne promenljive (log_revenue) izračunatu na trening setu, koja se primenjuje na sve instance u test setu.

Hide

```
baseline_mean_val = mean(train_set$log_revenue, na.rm = TRUE)

# Kreiranje vektora predikcija za test set (konstantna vrednost proseka)
baseline_preds = rep(baseline_mean_val, nrow(test_set_processed))
actuals = test_set_processed$log_revenue

print(paste("Izračunata bazna srednja vrednost za predikciju:", round(baseline_mean_val, 4)))
```

7.6.2 Izračunavanje MAE

Izračunavamo srednju apsolutnu grešku referentnog modela nad test setom kao meru prosečnog odstupanja apsolutnih vrednosti.

Hide

```
baseline_mae = mean(abs(actuals - baseline_preds))
print(paste("Baseline MAE:", round(baseline_mae, 4)))
```

7.6.3 Izračunavanje RMSE

Izračunavamo koren srednje kvadratne greške, čime dajemo veći značaj većim odstupanjima u predikciji.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

Hide

```
baseline_rmse = sqrt(mean((actuals - baseline_preds)^2))
print(paste("Baseline RMSE:", round(baseline_rmse, 4)))
```

7.6.4 Izračunavanje R-squared R^2

Izračunavamo koeficijent determinacije koji za prost prosek na test setu tipično ima vrednost blisku nuli ili negativnu, pošto model ne objašnjava varijansu.

Hide

```
# 4. Izračunavanje R-squared
ss_res = sum((actuals - baseline_preds)^2)
ss_tot = sum((actuals - mean(actuals))^2)
baseline_r2 = 1 - (ss_res / ss_tot)

print(paste("Baseline R-squared:", round(baseline_r2, 4)))
```

7.6.5 Pregledna tabela rezultata baseline modela

Sve izračunate metrike referentnog modela sumiramo u jedinstvenu tabelu radi lakšeg poređenja sa budućim naprednim modelima.

Hide

```
baseline_results_table = data.frame(
  Model = "Baseline (Mean Prediction)",
  R_squared = round(baseline_r2, 4),
  MAE = round(baseline_mae, 4),
  RMSE = round(baseline_rmse, 4)
)

knitr::kable(baseline_results_table, caption = "Performanse referentnog (baseline) modela na test setu")
```

Baseline model (R^2 približno 0, MAE = 0.554, RMSE = 0.6946) nema stvarnu prediktivnu moć i služi isključivo kao početna referenca, svaki napredniji model mora postići niže greške i pozitivan R^2

7.7 Linearna regresija

Linearna regresija predstavlja osnovni model sa kojim ćemo uporediti naprednije algoritme mašinskog učenja.

7.7.1 Treniranje modela i unakrsna validacija

Model linearne regresije treniramo nad pripremljenim trening skupom (train_set_processed), uz primenu definisane 5-fold unakrsne validacije. Iz prediktora izuzimamo originalnu kolonu sales_revenue_usd.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
set.seed(123)

# Treniranje linearne regresije korišćenjem 5-fold unakrsne validacije
lr_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "lm",
  trControl = train_control
)

# Prikaz rezultata unakrsne validacije za linearnu regresiju
print(lr_model)
```

Na osnovu rezultata unakrsne validacije, linearna regresija uspešno objašnjava oko 90% varijanse prihoda R^2 priblizno 0.90,sa prosečnom greškom predviđanja (RMSE) od 0.22. Ovi pokazatelji potvrđuju da model ima dobru stabilnost i da predstavlja dobru, pouzdanu polaznu osnovu za dalje poređenje sa naprednijim algoritmima.

7.7.2 Evaluacija modela na test setu

Nakon što je model treniran, vršimo predikciju nad potpuno izolovanim test skupom (test_set_processed) i upoređujemo njegove performanse sa referentnim (baseline) modelom.

Hide

```
lr_preds = predict(lr_model, newdata = test_set_processed)
actuals  = test_set_processed$log_revenue

# Izračunavanje metrika
lr_mae  = mean(abs(actuals - lr_preds))
lr_rmse = sqrt(mean((actuals - lr_preds)^2))

ss_res = sum((actuals - lr_preds)^2)
ss_tot = sum((actuals - mean(actuals))^2)
lr_r2  = 1 - (ss_res / ss_tot)

# Uporedna tabela performansi
lr_results_table = data.frame(
  Model = c("Baseline (Mean Prediction)", "Linear Regression"),
  R_squared = round(c(baseline_r2, lr_r2), 4),
  MAE = round(c(baseline_mae, lr_mae), 4),
  RMSE = round(c(baseline_rmse, lr_rmse), 4)
)

knitr::kable(lr_results_table, caption = "Poređenje performansi Baseline modela i Linearne regresi
je")
```

Linearna regresija drastično nadmašuje bazni model na izolovanom test setu, smanjujući MAE na 0.1710 i RMSE na 0.2181, uz visoki koeficijent determinacije R^2 = 0.9014. Model objašnjava preko 90% varijanse prihoda, što potvrđuje da izabrani skup od 13 prediktora nosi snažan linearni signal.

7.7.3 Interpretacija koeficijenata

Analizom vrednosti koeficijenata utvrđujemo smer i jačinu uticaja pojedinačnih prediktora na ciljnu promenljivu log_revenue.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
# Ekstrakcija koeficijenata iz modela
model_summary = summary(lr_model$finalModel)
coefs = model_summary$coefficients

coef_df = data.frame(
  Variable = rownames(coefs),
  Estimate = coefs[, 1],
  Std_Error = coefs[, 2],
  t_value = coefs[, 3],
  p_value = coefs[, 4]
)
rownames(coef_df) = NULL

# Prikaz top 10 najuticajnijih prediktora po apsolutnoj vrednosti koeficijenta
coef_df %>%
  filter(Variable != "(Intercept)") %>%
  arrange(desc(abs(Estimate))) %>%
  head(10) %>%
  knitr::kable(caption = "Top 10 koeficijenata linearne regresije")
```

Analizom koeficijenata uočava se da najveći negativan uticaj na prihod imaju novi i redovni kupci (customer_segment), dok kategorije Electronics i Home Appliances ostvaruju najjači pozitivan efekat. Od numeričkih prediktora ističe se log_budget, čijim rastom prihod stabilno raste.

7.7.4 Provera pretpostavki modela

Kako bi zaključci izvedeni iz linearne regresije bili statistički validni, grafički proveravamo osnovne pretpostavke modela: linearnost odnosa, homoscedastičnost (konstantnu varijansu reziduala) i normalnost njihove distribucije.

Hide

```
par(mfrow = c(2, 2))
plot(lr_model$finalModel)
par(mfrow = c(1, 1))
```

Residuals vs Fitted & Scale-Location:Ravnomerna raspršenost reziduala oko nule potvrđuje da su pretpostavke linearnosti i homoscedastičnosti zadovoljene za glavnu masu podataka.

Normal Q-Q: Reziduali u centralnom opsegu prate normalnu raspodelu, dok odstupanja na donjem repu ne ugrožavaju stabilnost modela usled velikog uzorka od 48.000 opservacija.

Residuals vs Leverage: Ne postoje kritično uticajne tačke koje bi narušile ocene parametara - svega nekoliko opservacija blago dodiruje granice Cookove udaljenosti.

7.8 Ridge i Lasso regresija

Regularizovane metode proširuju standardnu linearnu regresiju dodavanjem kazne na veličinu koeficijenata, čime sprečavamo preterano prilagođavanje (overfitting) i uspešno upravljamo prostorom prediktora

7.8.1 Priprema podataka za treniranje

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
x_train = model.matrix(log_revenue ~ . - sales_revenue_usd, data = train_set_processed)[, -1]
y_train = train_set_processed$log_revenue

x_test  = model.matrix(log_revenue ~ . - sales_revenue_usd, data = test_set_processed)[, -1]
y_test  = test_set_processed$log_revenue
```

7.8.2 Treniranje Ridge modela i unakrsna validacija za lambda_min

Za Ridge regresiju postavljamo parametar alpha = 0 (L2 regularizacija), dok optimalnu vrednost penalizacije (lambda.min) pronalazimo putem 5-fold unakrsne validacije preko funkcije cv.glmnet().

Hide

```
set.seed(123)

cv_ridge = cv.glmnet(x_train, y_train, alpha = 0, nfolds = 5)
best_lambda_ridge = cv_ridge$lambda.min

print(paste("Optimalna Ridge lambda (min):", round(best_lambda_ridge, 6)))

ridge_model = glmnet(x_train, y_train, alpha = 0, lambda = best_lambda_ridge)
```

7.8.3 Treniranje Lasso modela i unakrsna validacija zalambda_min

Za Lasso regresiju postavljamo parametar alpha = 1 (L1 regularizacija), čime pored smanjenja koeficijenata omogućavamo i automatsku selekciju feature-a. Optimalnu vrednost lambda ponovo određujemo pomoću 5-fold unakrsne validacije.

Hide

```
set.seed(123)

cv_lasso = cv.glmnet(x_train, y_train, alpha = 1, nfolds = 5)
best_lambda_lasso = cv_lasso$lambda.min

print(paste("Optimalna Lasso lambda (min):", round(best_lambda_lasso, 6)))

lasso_model = glmnet(x_train, y_train, alpha = 1, lambda = best_lambda_lasso)
```

7.8.4 Uporedna analiza performansi na test setu

Nakon obuke regularizovanih modela, vršimo predikciju nad test skupom i upoređujemo njihove performanse sa baznim modelom i standardnom linearnom regresijom.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
ridge_preds = predict(ridge_model, newx = x_test)
lasso_preds = predict(lasso_model, newx = x_test)

# Pomoćna funkcija za izračunavanje metrika
calc_metrics = function(actuals, preds) {
  mae = mean(abs(actuals - preds))
  rmse = sqrt(mean((actuals - preds)^2))
  ss_res = sum((actuals - preds)^2)
  ss_tot = sum((actuals - mean(actuals))^2)
  r2 = 1 - (ss_res / ss_tot)
  return(c(MAE = mae, RMSE = rmse, R_squared = r2))
}

ridge_m = calc_metrics(y_test, ridge_preds)
lasso_m = calc_metrics(y_test, lasso_preds)

models_comparison_table = data.frame(
  Model = c("Baseline (Mean Prediction)", "Linear Regression", "Ridge Regression", "Lasso Regressi
on"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"]), 4)

)

knitr::kable(models_comparison_table, caption = "Poređenje performansi linearnih modela (Baseline,
linearna regresija, Ridge, Lasso) na test setu")
```

Ridge regresija beleži blagi pad R^2 = 0.8933, zadržavajući visoku stabilnost, ali L2 penalizacija u ovom slučaju ne uspeva da nadmaši linearnu regresiju.

Lasso regresija ostvaruje identične performanse kao standardna linearna regresija, što sugeriše da model nije pretrpeo preterano prilagođavanje (overfitting) i da su izabrani prediktori već optimalno postavljeni.

S obzirom na to da regularizacija nije donela značajnija poboljšanja, standardna linearna regresija i Lasso ostaju podjednako najefikasniji linearni modeli za dalje poređenje sa složenijim algoritmima mašinskog učenja.

7.9 Stablo odlučivanja (Decision Tree)

Stablo odlučivanja predstavlja neparametrijski algoritam mašinskog učenja koji rekurzivno deli prostor feature-a na bazi pravila odlučivanja, omogućavajući hvatanje nelinearnih odnosa i interakcija između prediktora.

7.9.1 Treniranje i podešavanje parametara modela

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
set.seed(123)

# Definisanje grid-a za podešavanje cp (complexity parameter)
tree_grid = expand.grid(cp = seq(0.0001, 0.01, length = 10))

# Treniranje stabla odlučivanja uz 5-fold unakrsnu validaciju
tree_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "rpart",
  trControl = train_control,
  tuneGrid = tree_grid
)

# Prikaz rezultata unakrsne validacije za stablo odlučivanja
print(tree_model)
```

Kroz proces 5-fold unakrsne validacije testirano je 10 različitih vrednosti parametra složenosti (cp). Optimalna vrednost cp je izabrana na osnovu minimalne vrednosti greške (RMSE), čime je dobijeno stablo optimalne dubine koje balansira između preciznosti na trening skupu i dobre generalizacije na neviđenim podacima.

7.9.2 Evaluacija modela na test setu i poređenje

Nakon treniranja i podešavanja, vršimo predikciju nad potpuno izolovanim test skupom i upoređujemo performanse stabla odlučivanja sa prethodnim linearnim modelima.

Hide

```
# Predikcije na test setu
tree_preds = predict(tree_model, newdata = test_set_processed)

# Izračunavanje metrika za stablo
tree_m = calc_metrics(y_test, tree_preds)

# Uporedna tabela svih modela do sada
all_models_comparison = data.frame(
  Model = c("Baseline (Mean Prediction)", "Linear Regression", "Ridge Regression", "Lasso Regressi
on", "Decision Tree"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"], tree_m["R_sq
uared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"], tree_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"], tree_m["RMSE"]), 4)
)

knitr::kable(all_models_comparison, caption = "Poređenje performansi svih modela na test setu (ukl
jučujući Stablo odlučivanja)")
```

Sa R^2 = 0.8714 i nešto većim vrednostima grešaka, stablo odlučivanja zaostaje za linearnim modelima R^2 = 0.9014. Ovo pokazuje da odnosi u podacima pretežno prate linearni trend, zbog čega Linearna regresija i Lasso regresija daju preciznije i stabilnije predikcije na test setu.

7.10 Random Forest

Random Forest predstavlja moćan algoritam koji kombinuje veći broj stabala odlučivanja (bagging pristup), čime se efikasno smanjuje rizik od preteranog prilagođavanja (overfitting-u) i poboljšava stabilnost predikcija.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

7.10.1 Treniranje modela i podešavanje broja stabala

Hide

```
set.seed(123)

# Definisanje grid-a za podešavanje hiperparametara (mtry)
rf_grid = expand.grid(mtry = c(2,4))

# Treniranje Random Forest modela uz 5-fold unakrsnu validaciju
rf_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "rf",
  trControl = train_control,
  tuneGrid = rf_grid,
  ntree = 150
)

# Prikaz rezultata unakrsne validacije
print(rf_model)
```

Unakrsnom validacijom izabran je optimalni parametar mtry = 4, pri kome Random Forest postiže R^2 = 0.9053 i RMSE = 0.223. Agregacijom 150 stabala odlučivanja uspešno su prevaziđena ograničenja pojedinačnog stabla, čime je model objasnio preko 90.5% varijanse prihoda.

7.10.2 Evaluacija modela na test setu i poređenje

Nakon obuke, vršimo predikciju nad izolovanim test skupom i upoređujemo performanse Slučajnih šuma sa svim prethodnim modelima.

Hide

```
rf_preds = predict(rf_model, newdata = test_set_processed)

# Izračunavanje metrika za Random Forest
rf_m = calc_metrics(y_test, rf_preds)

# Proširena uporedna tabela svih modela
all_models_comparison_rf = data.frame(
  Model = c("Baseline", "Linear Regression", "Ridge", "Lasso", "Decision Tree", "Random Forest"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"], tree_m["R_sq
uared"], rf_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"], tree_m["MAE"], rf_m["MAE"]),
4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"], tree_m["RMSE"], rf_m["R
MSE"]), 4)

)
```

Random Forest ostvaruje do sada najbolje rezultate (R^2=0.9016 ,RMSE=0.2179), čime je nadmašio i pojedinačno stablo (R^2=0.8714) i sve linearne modele (R^2 = 0.9014). Agregacija 150 stabala omogućila je obuhvatanje složenijih obrazaca u podacima uz najnižu ukupnu grešku predikcije na test setu.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

7.11 Gradient Boosting

Gradient Boosting predstavlja napredni algoritam koji sekvencijalno gradi stabla odlučivanja, pri čemu svako novo stablo uči na greškama (rezidualima) prethodnih stabala. Ovaj pristup obično obezbeđuje izuzetnu prediktivnu moć.

7.11.1 Treniranje i podešavanje modela

Hide

```
set.seed(123)

# Definisanje manjeg grid-a za GBM kako bi se trening optimizovao
gbm_grid = expand.grid(
  interaction.depth = c(3, 5),
  n.trees = c(150),
  shrinkage = c(0.1),
  n.minobsinnode = c(10)
)

# Treniranje GBM modela uz 5-fold unakrsnu validaciju
gbm_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "gbm",
  trControl = train_control,
  tuneGrid = gbm_grid,
  verbose = FALSE
)

# Prikaz rezultata unakrsne validacije
print(gbm_model)
```

Na osnovu rezultata unakrsne validacije, Gradient Boosting uspešno objašnjava oko 93% varijanse prihoda R^2 priblizno 0.9328,sa prosečnom greškom predviđanja (RMSE) od RMSE približno 0.1831.

7.11.2 Evaluacija modela na test setu

Nakon obuke, vršimo predikciju nad test skupom i računamo evaluacione metrike.

Hide

```
gbm_preds = predict(gbm_model, newdata = test_set_processed)

# Izračunavanje metrika za GBM
gbm_m = calc_metrics(y_test, gbm_preds)
```

7.12 Konačno poređenje performansi svih modela

U završnoj fazi analize, sumiramo rezultate svih treniranih modela (od baznog modela, preko linearnih modela, stabla odlučivanja i slučajnih šuma, do Gradient Boosting-a) u jedinstvenu uporednu tabelu.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
# Kreiranje finalne uporedne tabele svih modela
final_models_comparison = data.frame(
  Model = c("Baseline", "Linear Regression", "Ridge", "Lasso", "Decision Tree", "Random Forest",
"GBM"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"], tree_m["R_sq
uared"], rf_m["R_squared"], gbm_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"], tree_m["MAE"], rf_m["MAE"],
gbm_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"], tree_m["RMSE"], rf_m["R
MSE"], gbm_m["RMSE"]), 4)
)

knitr::kable(final_models_comparison, caption = "Konačno poređenje performansi svih modela na test
```

Konačni rezultati potvrdili su jasnu hijerarhiju modela, gde Gradient Boosting (GBM) zauzima ubedljivo prvo mesto sa R^2 = 0.9302, MAE = 0.1453 i RMSE = 0.1835.Iako su linearna regresija i Lasso postavili dobre rezulatate (R^2 = 0.9014),gradient boosting algoritam je uspeo da dodatno smanji grešku predikcije za preko 14%. RMSE greška od 0.1835 na logaritamskoj skali odgovara relativnoj grešci predikcije prihoda u dolarima od približno 20%..Sposobnost sekvencijalnog ispravljanja grešaka čini GBM najpreciznijim rešenjem za predikciju prihoda.

7.13 Model Interpretation - Analiza reziduala

Nakon što smo kroz uporednu analizu utvrdili Gradient Boosting (GBM) daje najbolje i najstabilnije rezultate, sprovodimo detaljnu analizu reziduala (grešaka) nad izolovanim test skupom. Ova analiza nam omogućava da sagledamo gde model tačno greši, da li postoje sistematska odstupanja i kako se ponaša na ekstremnim vrednostima.

7.13.1 Residual vs Predicted (Analiza reziduala u odnosu na predikcije)

Prvi korak obuhvata grafičko poređenje predviđenih vrednosti i dobijenih reziduala (Actual - Predicted) na test setu

Hide

```
# Generisanje predikcija i reziduala na test setu za GBM model

gbm_preds = predict(gbm_model, newdata = test_set_processed)
gbm_residuals = test_set_processed$log_revenue - gbm_preds

res_df = data.frame(
  Actual = test_set_processed$log_revenue,
  Predicted = gbm_preds,
  Residuals = gbm_residuals
)

# Grafikon: Residuals vs Predicted (GBM)
ggplot(res_df, aes(x = Predicted, y = Residuals)) +
  geom_point(alpha = 0.4, color = "#2C3E50") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red", size = 1) +
  geom_smooth(method = "loess", se = FALSE, color = "darkorange", size = 1) +
  labs(title = "Residuals vs Predicted na test setu (GBM Model)",
       x = "Predviđene vrednosti (log_revenue)",
       y = "Reziduali (Stvarno - Predikcija)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11))
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js

U centralnom i najgušćem opsegu predikcija (log_revenue od 7.5 do 9.5), reziduali su ravnomerno raspršeni oko nule, a LOESS linija je potpuno ravna, što potvrđuje stabilnost i odsustvo pristrasnosti. Blagi rast linije na samom desnom rubu (> 10) posledica je manjeg broja opservacija sa ekstremno visokim prihodom.

7.13.2 Q-Q plot (Provera normalnosti distribucije reziduala)

Proveravamo da li greške modela prate normalnu raspodelu, što je ključno za validnost statističkih zaključaka.

Hide

```
ggplot(res_df, aes(sample = Residuals)) +
  stat_qq(color = "steelblue", alpha = 0.6) +
  stat_qq_line(color = "red", linetype = "dashed") +
  labs(title = "Normal Q-Q Plot reziduala na test setu",
       x = "Teoretski kvantili",
       y = "Uzorci kvantili (Reziduali)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11))
```

Reziduali GBM modela pokazuju izuzetno dobro poklapanje sa teorijskom linijom normalne raspodele duž gotovo celog raspona kvantila. Minimalna odstupanja na samim krajevima repova ne narušavaju opšti zaključak da su greške predikcije normalno distribuirane, što potvrdjuje visoku pouzdanost i stabilnost modela na test setu.

7.13.3 Analiza grešaka na ekstremnim opservacijama

U ovom koraku identifikujemo instance sa najvećim apsolutnim greškama predikcije kako bismo utvrdili da li model sistematski podbacuje kod specifičnih ekstremnih slučajeva.

Hide

```
res_df[, "Abs_Residuals"] = abs(res_df[, "Residuals"])
extreme_errors = res_df[order(-res_df[, "Abs_Residuals"]), ]

knitr::kable(
  head(extreme_errors[, c("Actual", "Predicted", "Residuals", "Abs_Residuals")], 5),
  caption = "Top 5 opservacija sa najvećom greškom predviđanja na test setu"
)
```

Najveća zabeležena apsolutna greška GBM modela iznosi svega 0.7105, što potvrđuje odsustvo drastičnih outlier-a u predikcijama. Greške se javljaju isključivo na ekstremnim rubovima raspodele - blago precenjivanje kod izuzetno niskih i umereno potcenjivanje kod izrazito visokih prihoda - što je uobičajena pojava pri modelovanju realnih ekonomskih podataka

7.13.4 Identifikacija sistematskih grešaka (Bias i rasipanje)

Kvantitativno proveravamo prosečnu pristrasnost (bias) modela na test setu, koja bi trebala da teži nuli.

Hide

```
mean_bias = mean(gbm_residuals)
print(paste("Prosečna greška (Bias) na test setu:", round(mean_bias, 6)))
```

Ovako mala vrednost bias-a potvrđuje da model nema pristrasnost na novim neviđenim podacima.

Loading [MathJax]/jax/output/HTML-CSS/jax.js

7.14 Feature Importance (Analiza značajnosti prediktora)

7.14.1 Random Forest importance

U prvoj fazi koristimo metodu baziranu na Random Forest-u da utvrdimo koje varijable unose najveće povećanje greške pri permutaciji tokom građenja stabala.

Hide

```
# Izračunavanje značajnosti varijabli za caret train model
importance_scores = varImp(rf_model, scale = FALSE)

# Pretvaranje u data frame
rf_imp_df = data.frame(
  Variable = rownames(importance_scores$importance),
  Importance = importance_scores$importance[, 1]
)

# Sortiranje u opadajućem redosledu
rf_imp_df = rf_imp_df[order(-rf_imp_df$Importance), ]

# Vizuelizacija top 10 najvažnijih varijabli
ggplot(head(rf_imp_df, 10), aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 10 najvažnijih varijabli (Random Forest)",
       x = "Varijable",
       y = "Značajnost") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11))
```

Na samom vrhu po značajnosti ubedljivo dominira logaritmovani budžet (log_budget), odmah praćen novim kupcima (customer_segmentNew) i ukupnim troškovima oglašavanja (total_ad_spend). Visok uticaj ostvaruju i redovni kupci (customer_segmentRegular), prodaja elektronike (product_categoryElectronics), kao i izvedene metrike potrošnje po predstavniku i promociji (spend_per_rep, spend_per_promotion). Ovakav raspored nedvosmisleno potvrđuje da visina finansijskih ulaganja i struktura kupaca predstavljaju glavne pokretače ostvarenog prihoda.

7.14.2 Permutation importance (Permutaciona važnost)

Kao dopunu Random Forest analizi, primenjujemo permutacionu važnost nad test setom kako bismo utvrdili stvarni pad performansi modela kada se vrednosti pojedinačnih varijabli nasumično promešaju.

Hide

Loading [MathJax]/jax/output/HTML-CSS/jax.js

```
# 1. Izračunavanje permutacione važnosti sa pred_wrapper-om
pfun = function(object, newdata) {
  predict(object, newdata = newdata)
}

perm_imp = vi(rf_model, method = "permute",
              train = test_set_processed,
              target = "log_revenue",
              metric = "RMSE",
              pred_wrapper = pfun)

# 2. Ručno crtanje preko ggplot2 (zaobilazi grešku sa xlim)
ggplot(head(perm_imp, 10), aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Permutaciona važnost varijabli (Test set)",
       x = "Varijable",
       y = "Povećanje greške (RMSE)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11))
```

Permutacija varijable customer_segment izaziva ubedljivo najveći porast RMSE greške (preko 0.35), dok izostavljanje prave strukture za product_category, log_budget i total_ad_spend takođe značajno narušava performanse. Ostali prediktori (season, sales_channel) imaju sekundarni uticaj na stabilnost predikcija.

7.14.3 Compare feature rankings (Poređenje rang-lista prediktora)

U završnom koraku upoređujemo rang-liste najvažnijih varijabli dobijene kroz Random Forest i Permutation importance analizu kako bismo potvrdili konzistentnost modela.

Hide

```
comparison_summary = data.frame(
  Rank = 1:5,
  Random_Forest = c("customer_segmentNew", "log_budget", "total_ad_spend", "customer_segmentRegula
r", "product_categoryElectronics"),
  Permutation_Importance = c("customer_segment", "product_category", "log_budget", "total_ad_spen
d", "season")
)

knitr::kable(
  comparison_summary,
  caption = "Poređenje vodećih prediktora po različitim metodama značajnosti"
)
```

Oba pristupa daju u potpunosti usaglašene i konzistentne rezultate. Dok Random Forest detaljnije izdvaja značaj pojedinačnih kategorija (poput novih kupaca ili elektronike), permutaciona analiza na nivou celokupnih varijabli potvrdjuje da su segment kupaca, kategorija proizvoda i obim marketinških ulaganja (log_budget, total_ad_spend) ključni pokretači modela. Drugim rečima, obe metode vode do istog poslovnog zaključka: na ukupne prihode najjače utiču profil kupca, vrsta robe i visina finansijskog uloga.

8 Zaključak

Loading [MathJax]/jax/output/HTML-CSS/jax.js

U ovom radu sprovedena je predikcija prihoda od prodaje (sales_revenue_usd / log_revenue) primenom modela mašinskog učenja nad Marketing & Sales skupom podataka (60.000 opservacija). Proces je obuhvatio pripremu podataka, MICE/PMM imputaciju, eksplorativnu analizu, inženjering obeležja i evaluaciju više klasa regresionih modela.

Analiza je pokazala da su ključni pokretači prihoda segment kupaca (naročito novi kupci), kategorija proizvoda (elektronika) i obim budžetskih ulaganja (log_budget, total_ad_spend). Kreiranjem novih promenljivih uspešno su izdvojeni ključni poslovni faktori i smanjena multikolinearnost.

Poređenje modela pokazalo je da je Gradient Boosting (GBM) ostvario ubedljivo najbolje rezultate ($R^2 = 0.9302$, RMSE = 0.1835), nadmašivši linearnu regresiju ($R^2 = 0.9014$) i Random Forest ($R^2 = 0.9016$). Sekvencijalno učenje na greškama omogućilo je GBM-u da obuhvati složene nelinearne interakcije uz zadržavanje visoke pouzdanosti na test setu.

Ovaj rad potvrđuje da kombinacija kvalitetne pripreme podataka, inženjeringa obeležja i naprednih boosting algoritama predstavlja izuzetno pouzdan pristup za donošenje informisanih poslovnih i finansijskih odluka.

Loading [MathJax]/jax/output/HTML-CSS/jax.js